密级：公开

南昌大学

NANCHANG UNIVERSITY

学 士 学 位 论 文

THESIS OF BACHELOR

（2022 —2026 年）

题　　目　基于 RAG 架构的 AI 知识库系统

学　　院：软件学院　　系　软件工程

专业班级：卓越 2201 班

学生姓名：黄逸飞　　学号：8008122113

指导教师：林德钰　　职称：讲师

起讫日期：2025 年 12 月 7 日至 2026 年 5 月 31 日

南 昌 大 学

学士学位论文原创性申明

本人郑重申明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式表明。本人完全意识到本申明的法律后果由本人承担。

作者签名：　　日期：

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权南昌大学可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

保密☑，在　年解密后适用本授权书。

本学位论文属于

不保密□。

（请在以上相应方框内打“√”）

作者签名：　　日期：

导师签名：　　日期：

基于 RAG 架构的 AI 知识库系统

专　　业：　　软件工程　　学　　号：　8008122113
学生姓名：　黄逸飞　　指导教师：　林德钰

摘要

随着企业数字化建设的不断推进，企业内部积累了大量技术文档、业务资料、操作截图、扫描件和问题排查记录等知识资源。然而，传统知识管理方式主要依赖文件夹归档和关键字检索，存在知识分散、检索效率不高、图片类资料难以利用以及知识复用能力不足等问题。针对上述情况，本文设计并实现了一套基于 OCR 与 RAG 技术的企业智能知识库系统，旨在提升企业知识资料的统一管理、语义检索和智能问答能力。

本文首先分析了企业知识管理的实际需求，并结合多模态资料处理和检索增强生成技术，确定了系统的总体架构和功能模块。系统采用 Spring Boot 作为后端开发框架，Vue 3 与 TypeScript 构建前端页面，使用 MySQL 存储用户、文件、对话、套餐等业务数据，使用 Redis 缓存短期会话和任务状态，使用 MinIO 存储文档、图片和扫描件等原始文件，使用 Kafka 对文档解析、OCR 识别、文本切片和向量化入库等耗时任务进行异步处理。系统通过 OCR 技术提取图片、截图和扫描件中的文字内容，并结合 Embedding 模型将文档解析结果和 OCR 识别结果转化为向量表示，存入 Elasticsearch 中，实现关键词检索与语义检索相结合的混合检索方式。

在智能问答方面，系统基于 RAG 技术构建问答流程。当用户提出问题后，系统先从知识库中检索相关文档片段或 OCR 识别内容，再将检索结果作为上下文交给大语言模型生成回答，从而提高回答与企业已有资料的关联性。同时，系统支持 WebSocket 流式输出、历史对话保存、引用来源展示和权限控制等功能，使用户能够较为方便地查看回答依据和历史记录。针对系统使用场景，本文还设计了套餐购买与微信支付模块，用于支持用户套餐权益和使用额度管理。

通过系统实现与测试结果表明，本文设计的企业智能知识库系统能够较好地完成文档、图片和扫描资料的统一管理，实现 OCR 识别、语义检索和智能问答等功能，在一定程度上提高了企业知识检索和复用效率。该系统为 RAG 技术、OCR 技术和多模态资料处理在企业知识管理场景中的应用提供了一种可行的实现方案。

关键词：企业知识库；RAG；OCR；多模态；语义检索；Elasticsearch；智能问答

AI knowledge base system based on RAG architecture

Abstract

With the continuous advancement of enterprise digitalization, a wealth of knowledge resources such as technical documents, business materials, operation screenshots, scanned copies, and problem troubleshooting records have accumulated within the enterprise. However, traditional knowledge management methods primarily rely on folder archiving and keyword retrieval, which present issues such as knowledge dispersion, low retrieval efficiency, difficulty in utilizing image-based materials, and insufficient knowledge reuse capabilities. In response to the above situation, this paper designs and implements an enterprise intelligent knowledge base system based on OCR and RAG technology, aiming to enhance the unified management, semantic retrieval, and intelligent question-answering capabilities of enterprise knowledge materials.

This article first analyzes the actual needs of enterprise knowledge management and, combining multimodal data processing and retrieval-enhanced generation technology, determines the overall architecture and functional modules of the system. The system uses Spring Boot as the backend development framework, Vue 3 and TypeScript to build frontend pages, MySQL to store business data such as users, files, conversations, and packages, Redis to cache short-term sessions and task statuses, MinIO to store original files such as documents, images, and scanned documents, and Kafka to asynchronously process time-consuming tasks such as document parsing, OCR recognition, text slicing, and vectorization for storage. The system extracts text content from images, screenshots, and scanned documents through OCR technology, and combines Embedding models to convert document parsing results and OCR recognition results into vector representations, which are stored in Elasticsearch to achieve a hybrid retrieval method combining keyword retrieval and semantic retrieval.

In terms of intelligent question answering, the system constructs a question-answering process based on RAG technology. When a user poses a question, the system first retrieves relevant document fragments or OCR recognition content from the knowledge base, and then uses the retrieval results as context to generate an answer from a large language model, thereby improving the relevance of the answer to the enterprise's existing materials. At the same time, the system supports WebSocket streaming output, history conversation saving, reference source display, and permission control, enabling users to easily view the basis for the answer and historical records. For the system usage scenarios, this paper also designs a package purchase and WeChat payment module to suppo

rt user package benefits and usage quota management.

The system implementation and test results demonstrate that the enterprise intelligent knowledge base system designed in this paper can effectively manage documents, images, and scanned materials in a unified manner. It achieves functions such as OCR recognition, semantic retrieval, and intelligent question answering, thereby enhancing the efficiency of enterprise knowledge retrieval and reuse to a certain extent. This system provides a feasible implementation solution for the application of RAG technology, OCR technology, and multimodal data processing in enterprise knowledge management scenarios.

Keywords: enterprise knowledge base; RAG; OCR; multimodal; semantic search; Elasticsearch; Intelligent Q&A

目录

摘要 …… Ⅰ
Abstract …… Ⅱ
第一章 绪论……1
1 .1 目背景及意义 ……1
1. 2 国内外研究现状 ……2
1. 3 论文研究内容 ……4
1. 4 论文组织结构 ……13
第二章 相关技术……31
2. 1 项目选用的技术简介……31
2. 2 相关技术选型对比研究……32
第三章 系统分析……31
3. 1 系统业务需求……31
3. 2 系统功能需求……32
3. 3 系统非功能性需求……32
3. 4 系可行性分析……32
第四章 系统设计……31
4. 1 系统总体设计……31
4. 2 数据库设计……32
4. 3 系统功能设计……32
4. 4 系统界面设计（可选）……32
第五章 系统实现……31
5. 1 系统实现概述……31
5. 2 功能模块实现……32
第六章 系统测试……31
6. 1 测试策略与方法……31
6. 2 测试用例与结果……32
第七章 总结与展望 ……31
7. 1 工作总结……31
7. 2 未来展望……32
参考文献（References）…… 138
致谢 …… 150

第一章 绪论

1.1 项目背景及意义

我申请的项目的核心意义是改善企业知识管理中效率低、检索不便和知识分散的问题。在外部公司恒生电子实习期间，我发现公司内部积累了大量的技术信息，包括系统接口规范、业务组件用户手册、配置参数规范、开发标准、问题调查记录、项目经验总结，以及一些截图、扫描件和操作说明图片。虽然这些材料在一定程度上已经存档和分类，但它们仍然主要存储在传统的文件夹或文档系统中，文档之间缺乏有效的关联，图像和扫描材料难以直接检索。

这种方法在实际应用中有一定的局限性。一方面，技术信息分布在不同的项目团队和系统模块之间，这很容易导致知识分散。新员工或跨项目合作者通常需要花更多的时间搜索信息。另一方面，企业中的大量知识不仅存在于文本文档中，还分布在系统截图、错误截图、配置界面和扫描文档中。这类信息包含一定的价值，但在传统的管理方法下很难统一检索和重用。

我在个人学习和实习过程中也遇到了类似的问题。无论是使用在线文档做笔记还是根据目录组织信息，都存在后期参考效率不高、知识点之间的联系不明确的情况。特别是当面对大量分散的内容时，传统的笔记或文档系统发现很难根据具体问题快速找到答案；对于图像注释、屏幕截图和扫描材料，通常需要手动阅读和整理。

后来，实习主管提出了将现有的文档知识库升级为人工智能知识库的想法。经过多次交流，我逐渐意识到，将大规模语言模型、OCR 文本识别能力和内部知识库相结合，可以在一定程度上增强知识检索和服务能力。基于大型语言模型的检索增强生成(retrieval-augmented generation,RAG)正在成为工业知识分析和决策支持的一种有前途的方法[4]。与仅依赖关键字搜索相比，基于 RAG 的系统可以首先从企业知识库中检索相关内容，然后结合大型模型生成答案；OCR 可以从图像、扫描文档、屏幕截图和其他材料中提取文本，使他们能够参与后续的搜索和问答。

基于这一背景，我明确了本项目的建设方向：为企业场景构建多模态 RAG 智能知识库系统，实现企业文档、图像和扫描材料的统一管理、OCR 识别、语义检索、智能问答和知识辅助生成。该系统采用 Spring Boot、Vue 3、MySQL、Redis、Elasticsearch、Kafka 和 MinIO 等技术，支持文档上传、图像上传、文本解析、OCR 识别、切片、矢量化和索引，将分散的非结构化数据转化为可检索和问答知识资源。在索引阶段，需要使用分块技术将文本分割成不同的块[12]。

在功能设计方面，系统支持 Word、PDF、TXT 等常见文档处理，还支持图像、截图和扫描件的 OCR 识别。系统将对解析或识别的文本进行切片和嵌入，对其进行矢量化，并将其存储在 Elasticsearch 中。然后，它将结合关键字检索和语义检索来提高搜索性能。用户可以通过自然语言提问，系统根据相关文档内容或 OCR 识别结果生成答案，从而减少手动搜索和信息过滤时间。

此外，该系统还增加了文件分片上传、异步文档处理、流对话、历史记录保存和参考源显示等功能。我们的方法引入了后期分块技术，利用长上下文嵌入生成灵活的、可变长度的文本片段，并采用高斯混合模型（GMM）来实现软概率聚类[10]。通过 Kafka 处理文档解析、OCR 识别和矢量化仓储等任务，可以减少耗时操作对用户体验的影响；通过 Redis 和 MySQL 分别处理短期上下文和长期历史，可以平衡对话效率和数据持久性。对于 OCR 识别的内容，该系统也将被纳入权限控制范围，以尽可能确保知识共享和数据安全之间的平衡。

总体而言，该项目侧重于企业知识管理场景，探索如何将 RAG、OCR 和多模式数据处理功能相结合，以提高文档管理、知识检索和体验重用效率。通过该系统，可以将传统文档、图像、截图和扫描件等各种材料集成到知识库管理中，帮助用户轻松获取所需信息，为企业知识积累、共享和应用提供可行的智能解决方案。

1.2 国内外研究现状

1、国外研究现状

国外对企业知识管理的研究起步较早，最初主要集中在知识的收集、分类、存储、共享和复用上。随着企业信息化水平的不断提升，企业内部逐渐积累了大量的文档数据、项目经验、业务流程、技术规范和问题解决记录。传统知识管理系统已从简单的文档归档系统演变为支持全文搜索、权限管理、标签分类和协同编辑的企业知识平台。然而，传统知识库仍然主要依赖人工分类和关键词搜索。当面对大量非结构化文档时，容易出现检索不准确、知识关联性弱以及用户搜索成本高等问题。

近年来，随着大型语言模型的发展，国外的研究界和工业界逐渐将人工智能技术引入知识管理领域。其中，检索增强生成（RAG）已成为构建智能知识库系统的重要技术方法。RAG 由 Patrick Lewis 等人于 2020 年首次提出，其核心思想是将大型语言模型的参数化知识与外部知识库的非参数化知识相结合。在生成答案之前，会从外部知识源中检索相关内容，然后将检索结果作为上下文提供给模型以生成响应，从而增强模型在知识密集型任务中的准确性和可解释性。这种方

法可以在一定程度上缓解大型语言模型知识更新滞后、专业领域知识不足以及生成内容可能存在“虚幻”等问题。

在企业应用层面，许多国外科技公司和研究机构已将检索增强生成（RAG）应用于企业智能问答、客户支持、内部知识检索和自动化文档处理等场景。与传统关键词搜索相比，RAG 系统能够基于语义理解用户问题，并从企业私有文档、产品手册、历史工单和政策文件等数据源中检索相关内容。然后，它们结合大型语言模型提供自然语言答案。这种方法不仅提高了知识获取的效率，还使企业内部知识能够以更接近日常问答交互的形式被调用。LangChain 等开源框架也将 RAG 视为基于私有数据构建问答系统的重要解决方案。根据其官方文档，RAG 的根本目标是通过检索外部上下文信息来弥补大型语言模型上下文长度有限和训练知识静态性的不足。

与此同时，国外的研究已开始从“基础检索增强生成（RAG）”向“高级检索增强生成（RAG）”发展。例如，微软研究院提出了 GraphRAG，该技术结合了知识图谱、文本提取、网络分析以及大型语言模型的摘要能力，以更好地理解和组织私有文本数据。与仅依赖文本切片和向量检索的 RAG 相比，GraphRAG 更强调文档关系建模和全局信息摘要，使其适用于处理企业内部大量复杂、相互关联的非结构化文档。微软的 GraphRAG 文档还指出，该方法从原始文本中提取知识图谱，构建层次化的社区结构，并基于这些结构辅助 RAG 任务。

总体而言，国外人工智能知识库领域的研究与应用已从单一文档检索发展为涵盖语义检索、向量数据库、知识图谱、智能问答、文档自动化以及企业级代理的综合性方向。研究主要集中在以下几个方面：首先，如何提高检索准确率和召回率；其次，如何减少大型模型生成内容的幻觉问题，在处理需要事实核查、时效性强或领域特定的任务时，易出现知识缺失、信息幻觉等问题[14]；第三，如何实现企业私有数据的安全访问和权限控制；第四，如何为模型答案提供源引用，以增强结果的可信度；第五，如何将知识库与现有的企业系统（如 CRM、ERP、工单系统和文档管理系统）进行集成。

2、国内研究现状

国内企业知识管理系统的开发也经历了一个演变过程，从传统的文档管理和数字知识库过渡到智能知识库。在早期阶段，企业知识库主要用于集中文档存储、机构文档归档、项目数据管理和员工经验积累。系统的功能主要包括目录分类、关键词搜索、权限控制和版本管理。随着企业数字化转型的深入，内部知识量迅速增长，知识分散在不同的系统、项目团队和业务部门。传统知识库逐渐无法满

足高效检索和智能应用的需求。

近年来，随着国内大模型和智能应用平台的发展，“大模型+知识库”逐渐成为企业级人工智能实施的重要方向。相关行业报告指出，知识库是企业经验、流程、文档、政策和业务知识的集中载体，在提高员工生产力和促进组织协作方面发挥着重要作用。大模型、知识图谱、检索增强生成（RAG）等技术的结合，已逐渐将企业知识应用从传统检索转向自然语言交互和智能问答。特别是在金融、政务和电信等知识密集型行业，智能知识库已开始应用于智能客服、政策问答、业务咨询、智能报告生成和内部员工办公辅助等场景。

在技术实现方面，传统依赖专家经验或静态解析的方法已难以满足现代网络研究、运维与安全分析对效率、可解释性及可扩展性的需求[1]。目前中国较为常见的 AI 知识库解决方案通常包括文档解析、文本分割、向量化表示、向量数据库存储、相似性检索、提示词构建以及大型模型生成。具体而言，优化后的提示有助于有效进行需求工程，从而帮助利益相关者将不明确的想法转化为复杂的软件[17]。企业可以将 PDF、Word 文档、Markdown 文件、网页、接口文档和系统规范等非结构化数据导入知识库系统。经过预处理后，这些数据被转换为可检索的知识片段。当用户提出问题时，系统首先通过向量检索或混合检索找到相关的文档片段，然后将检索到的内容与用户的问题一起输入到大型语言模型中。模型生成最终答案。这种方法可以更好地适应企业内部知识分散、格式多样以及频繁更新的特点。

该领域的国内应用研究正逐渐从“能够回答问题”向“提供准确、可追溯且可管理的答案”转变。对于企业内部的知识库而言，仅仅集成大型模型并不能完全解决问题。系统还需考虑文档源管理、知识更新机制、权限控制、答案引用、检索质量评估以及敏感信息保护等问题。特别是在金融、政务、通信和工业制造等领域，知识内容往往具有强烈的专业性和安全性要求。因此，智能知识库系统不仅需要具备自然语言问答能力，还需确保答案内容的来源可靠、权限边界清晰，且数据处理过程可控。

从当前国内的应用情况来看，企业在实施人工智能（AI）知识库方面仍存在一些不足。首先，一些系统仍停留在“文档上传+向量检索+大型模型问答”的简单阶段，缺乏对文档结构、业务关系和知识层次的深入处理。其次，不同类型文档的解析效果差异较大。例如，表格、图像、代码块和接口参数描述等内容在分割和向量化过程中往往会丢失上下文信息。第三，知识库问答结果的准确性高度依赖于检索质量。如果检索到的片段不相关或缺乏足够的上下文，大型模型容易生成不完整甚至错误的答案。此外，企业内部知识往往涉及权限控制和数据安全。

如何在确保安全性和合规性的同时实现智能检索和问答，也是当前系统建设中需要解决的关键问题。

1.3 论文研究内容

论文围绕一个企业级智能知识管理系统展开研究与实现，重点研究如何利用 RAG，即检索增强生成技术，提升企业内部知识的管理、检索与问答能力。系统以企业文档、知识库资料等非结构化数据为基础，通过文件上传、解析、切分、向量化、索引构建和语义检索等流程，将传统静态知识转化为可被大模型理解和调用的知识资源。论文主要研究内容包括：

1、RAG 架构在企业知识管理场景中的应用

分析传统知识管理系统在检索准确性、语义理解能力、知识利用效率等方面的不足，引入“检索 + 生成”的 RAG 技术路线。系统在用户提问时，先从知识库中召回相关内容，再将检索结果与用户问题共同输入大模型，从而生成更贴近企业私有知识的回答。

2、企业知识库的数据处理流程设计

研究文档从上传到可检索的完整处理链路，包括文件存储、文本解析、内容分段、元数据管理、索引构建等。系统结合 MinIO、Kafka、Elasticsearch 等组件，实现文件的异步处理与高效检索，为后续智能问答提供知识基础。

3、智能问答与上下文管理机制

设计面向企业用户的智能问答模块，支持基于知识库的语义检索、上下文对话、历史记录保存以及引用来源展示。系统使用 Redis 管理短期会话上下文，使用 MySQL 持久化用户、知识库、聊天历史等核心数据，保证问答过程既具备连续性，又具备可追溯性。

4、多租户与权限管理设计

针对企业级应用场景，研究组织、用户、角色与知识资源之间的权限关系。系统通过组织标识、用户角色和访问控制机制，实现不同企业、部门或用户之间的数据隔离与权限管理，保障知识资产的安全性。

5、系统工程实现与验证

论文还包括系统整体架构设计与功能实现。后端基于 Spring Boot，前端基于 Vue 3 和 TypeScript，数据库采用 MySQL，缓存采用 Redis，检索服务采用 Elasticsearch，异步任务通过 Kafka 处理，对象存储使用 MinIO。通过系统功能测试与运行验证，证明该方案能够支持企业知识上传、管理、检索、智能问答和

历史追踪等核心需求。

6、基于 OCR 结果的多模态知识检索与增强生成机制

传统检索增强生成(RAG)方法主要面向纯文本场景，其检索与生成机制难以有效建模多模态文档中普遍存在的视觉元素、空间布局与结构语义，在图文混合、长文档及跨文档推理任务中表现受限[2]。在问答过程中，系统不仅可以检索普通文本内容，也可以召回由图片、扫描件和图片型 PDF 中识别出的 OCR 文本内容。用户提出问题后，系统基于 Elasticsearch 等检索能力匹配相关知识片段，并将检索结果作为上下文输入大模型，实现基于图文资料内容的智能问答。该机制扩展了 RAG 系统的知识覆盖范围，使原本难以被检索和利用的图像化资料也能参与智能问答。

7、提示词工程

本项目的提示词工程主要服务于 RAG 问答场景，核心目标是让大模型基于企业知识库检索结果进行可靠、可追溯的回答。

系统采用“系统规则 + 检索上下文 + 对话历史 + 用户问题”的组合式提示词结构。首先在 application.yml 中配置统一的 system 规则，约束模型使用简体中文回答、先给结论再给论据、引用知识库资料时标注来源编号和文件名，并在检索结果不足时回答“暂无相关信息”，避免模型凭空编造。

在生成回答前，后端会先根据用户问题执行混合检索，得到相关文档片段。随后将检索结果整理为带编号的上下文，例如 [1] 文件名 | 页码 | 内容片段，再用 <<REF>> 和 <<END>> 作为参考资料边界，插入到 system prompt 中。这样可以明确区分“系统指令”“知识库参考内容”和“用户输入”，降低提示词注入和上下文混淆风险。

项目还保留最近的对话历史，使模型能够理解多轮对话语境。同时系统会维护引用编号与文件 MD5、页码、锚点文本、命中片段之间的映射关系，使回答中的来源编号可以回溯到具体文档位置，支持前端引用预览。因此，提示词优化在 RAG 中呈现出“双重作用”：一方面用于增强检索端的精确性与召回率，另一方面用于提升生成端的连贯性与真实性[18]。

整体来看，本项目的提示词工程特点是：规则约束明确、检索内容结构化、回答格式可控、引用可追溯，并通过“无检索结果”规则抑制幻觉生成，适合企业知识库问答系统对准确性和可信度的要求。

第二章 相关技术

2.1 项目选用的技术简介

本项目是一个面向企业知识管理场景的多模态 RAG 智能知识库系统，主要采用 Spring Boot、Vue 3、MySQL、Redis、Elasticsearch、Kafka、MinIO、OCR、Embedding 向量化、大语言模型、WebSocket 和微信支付等技术，实现文档管理、图片识别、语义检索、智能问答、套餐购买等功能。

1、Spring Boot

Spring Boot 是基于 Spring 生态的 Java 后端开发框架，具有自动配置、快速开发和生态完善等特点。本项目使用 Spring Boot 构建后端服务，负责用户认证、知识库管理、文件上传、OCR 识别任务调度、RAG 问答、套餐管理和支付回调等核心业务逻辑。

2、Vue 3

Vue 3 是一种渐进式前端框架，适合构建交互式 Web 应用。项目使用 Vue 3 和 TypeScript 开发前端页面，实现用户登录、知识库管理、文档上传、图片上传、智能问答、历史记录查看、套餐购买等功能。Vue 3 的组件化开发方式有利于提升前端代码的复用性和维护性。

3、MySQL

MySQL 是常用的关系型数据库，适合存储结构化业务数据。本项目使用 MySQL 保存用户信息、文件元数据、知识库信息、对话记录、套餐订单、支付记录、组织权限等数据，作为系统的主要持久化存储。

4、Redis

Redis 是基于内存的高性能键值数据库，常用于缓存、会话管理和临时状态存储。本项目使用 Redis 保存短期对话上下文、验证码、登录状态、文件分片上传状态、生成过程状态等信息，从而提高系统响应速度，并减轻数据库访问压力。将句子嵌入分解为潜在主题，对主题相关的句子（包括非相邻句子）进行分组，同时保留上下文的丰富性[11]。

5、Elasticsearch

ElasticSearch 是一个分布式搜索与分析引擎，它建立在 Elastic Stack 之上，能够高效、实时地处理庞大的数据集，适用于所有类型的数据处理，包括文本、数字、日期和时间、地理空间、日志和事件、图像和二进制数据等，不仅具备处理结构化数据的能力，还能轻松应对非结构化数据[5]。Elasticsearch 是分布式搜索引擎，支持全文检索、结构化查询和向量检索。本项目使用 Elasticsearch 存

储文档切片后的文本内容和向量数据，并结合关键词检索与语义检索实现混合搜索，提高用户问题与知识内容之间的匹配效果。该方法集成了多种 ES 特有的技术,包括灵活的嵌套和连接类型的数据建模,用于统计分析的高级聚合框架,以及系统的查询和索引优化[3]。

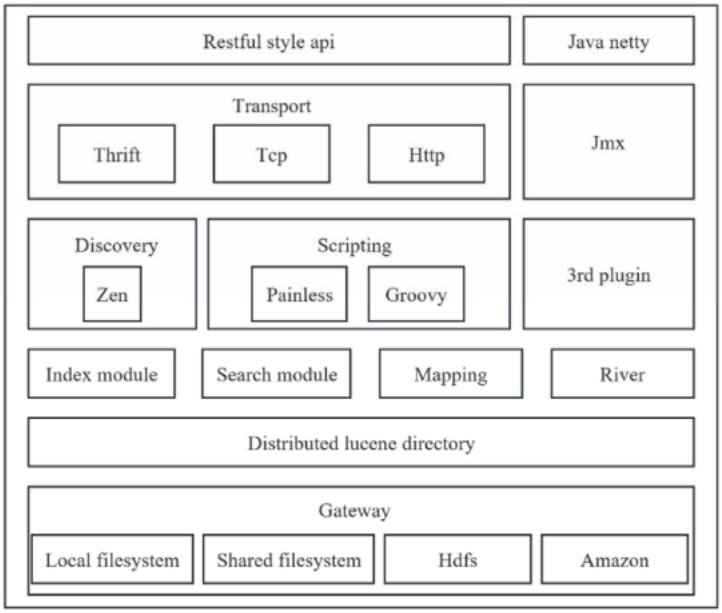

图 2-1　ElasticSearch 整体架构图

6、Kafka

Kafka 是一种高吞吐量的分布式消息队列，适合处理异步任务。本项目使用 Kafka 对文件解析、OCR 识别、文本切片、向量化入库等耗时任务进行异步处理，避免用户上传文件后长时间等待，提高系统的稳定性和可扩展性。

7、MinIO

MinIO 是兼容 S3 协议的对象存储系统，适合保存图片、文档、音视频等非结构化文件。本项目使用 MinIO 存储用户上传的 Word、PDF、TXT、图片、扫描件等原始资料，并为后续 OCR 识别、文本解析和来源追溯提供文件基础。

8、OCR 文字识别

OCR，即光学字符识别技术，用于从图片、截图、扫描件等图像资料中提取文字内容。本项目引入 OCR 能力，使系统不仅能够处理普通文本文档，也能够识别报错截图、配置截图、扫描文档和操作说明图片中的文字信息，从而扩展知识库的数据来源。

9、Embedding 向量化

Embedding 向量化技术可以将文本内容转化为向量表示，使计算机能够从语义层面比较文本之间的相似度。本项目将文档解析结果和 OCR 识别结果进行切

片后，通过 Embedding 模型生成向量，并存入 Elasticsearch，用于后续语义检索。

10、RAG 检索增强生成

RAG，即检索增强生成，是将信息检索与大语言模型生成能力结合的一种技术方案。本项目在用户提问后，先从知识库中检索相关文档片段，再将检索结果作为上下文交给大语言模型生成回答。该方式能够使回答更贴近企业内部资料，并提高结果的可追溯性。

11、大语言模型

大语言模型用于理解用户问题并生成自然语言回答。本项目将大语言模型与企业知识库结合，使系统能够根据用户问题、历史上下文和检索到的知识片段生成较为准确的回答，支持智能问答、内容总结和知识辅助生成等功能。当信息在 LLM 的上下文窗口内时，这些模型能够生成连贯且与上下文相关的答案[20]。

12、WebSocket

WebSocket 是一种支持客户端与服务端双向通信的网络协议，适合实时交互场景。本项目使用 WebSocket 实现智能问答的流式输出，使用户能够实时看到回答生成过程，同时支持停止生成、断线重连和对话状态恢复等功能。

13、微信支付

微信支付是常见的在线支付方式，适合应用于会员套餐、额度充值和服务购买等场景。本项目在套餐购买模块中接入微信扫码支付能力，用户可以通过微信完成套餐购买，系统根据支付结果更新用户套餐状态、使用额度和相关权限。

14、Spring Security 与 JWT

Spring Security 是 Spring 生态中的安全框架，JWT 作为一种轻量级的、自包含的、用于在前后端之间安全传输信息的简洁的、安全的令牌标准，因其无需在服务器端保存会话状态、易于扩展、跨域访问等优势，在前后端分离项目中得到了广泛应用[19]。本项目结合 Spring Security 和 JWT 实现用户登录认证、接口访问控制和权限校验，保证不同用户只能访问其授权范围内的知识库内容和系统功能。

2.2 相关技术选型对比研究

本项目面向企业知识管理和多模态 RAG 问答场景，需要同时满足文档管理、OCR 识别、语义检索、智能问答、异步处理、权限控制和套餐支付等需求。因此，在技术选型时主要从系统稳定性、开发成本、生态成熟度、可扩展性和与 Java 技术栈的适配程度等方面进行综合考虑。

1、RAG 与智能体编排技术选型

技术方案	优点	不足	本项目选择情况
自定义 RAG 流程	可控性高，便于结合现有业务、权限、OCR、文件来源追溯	需要自行实现检索、拼接上下文、调用模型等流程	当前主要采用
LangChain4j	Java 生态中的 LLM 应用框架，提供 RAG、Embedding、Memory、Tools 等能力	深度定制时仍需结合业务改造	可作为后续增强方案
Spring AI	与 Spring 生态结合较好，适合 Spring Boot 项目接入大模型能力	部分高级能力仍需自行封装	可用于规范化模型调用
LangGraph4j	支持状态图、多智能体、循环流程、断点和可视化，适合复杂 Agent 工作流	对简单 RAG 系统来说引入成本较高	适合后续扩展多智能体流程

表 2-2 RAG 与智能体编排技术选型

当前项目主要聚焦于一个定制的检索增强生成（RAG）流程，该流程包括“文档/OCR 内容摄取→文本切片→向量化→Elasticsearch 检索→大型模型生成响应”。这种方法有助于与系统现有的组织权限、引用源、历史记录和多模态数据处理逻辑进行集成。

对于 LangGraph4j 而言，此类框架更适合处理复杂的代理工作流程，如多轮任务规划、工具调用、人工确认和多代理协作。由于该项目目前的重点是放在企业知识库检索和问题回答上，而非复杂的代理自动化任务，因此 LangGraph4j 并未被用作核心实现框架。然而，在后续扩展中，可以考虑将“问题理解、OCR 内容分析、知识检索、答案生成和结果验证”等步骤整理成状态图流程。

2、检索与向量存储选型

技术方案	优点	不足	适用场景
Elasticsearch	支持全文检索、关键词搜索、过滤查询和向量检索，适合混合检索	向量检索能力相比专用向量库略弱	企业文档检索、关键词 + 语义混合搜索
Milvus	专业向量数据库，适合大规模向量检索	关键词检索和业务过滤需要额外组合	大规模向量召回

PostgreSQL + pgvector	架构简单，关系数据和向量数据可放在同一数据库	大规模检索性能有限	中小规模向量检索
MySQL 全文索引	使用简单，维护成本低	语义检索能力不足	普通关键词检索

表 2-3 检索与向量存储选型

本项目选择 Elasticsearch。原因是企业知识库检索不仅需要语义相似度，还需要关键词匹配、文件名过滤、组织权限过滤、来源信息查询等能力。Elasticsearch 同时支持全文检索和向量检索，比较适合实现“关键词 + 语义”的混合检索。

3、文件存储技术选型

技术方案	优点	不足	适用场景
MinIO	兼容 S3 协议，适合存储文档、图片、扫描件等非结构化文件	需要单独部署和维护	企业私有化文件存储
本地文件系统	实现简单，开发成本低	扩展性和可靠性较弱	单机测试或小型系统
云对象存储	稳定性高，运维成本低	依赖云厂商，有成本和数据合规问题	公有云部署系统
数据库存储文件	查询统一	大文件会影响数据库性能	少量小文件存储

表 2-4 文件存储技术选型

本项目选择 MinIO 存储用户上传的文档、图片、截图和扫描件。MinIO 适合保存非结构化文件，也便于后续根据引用来源回溯原始资料。

4、异步任务技术选型

技术方案	优点	不足	适用场景
Kafka	吞吐量高，支持分区、副本和持久化，适合大规模异步处理	部署和维护成本较高	文件解析、OCR、向量化等异步任务
RabbitMQ	路由灵活，使用简单	高吞吐场景弱于 Kafka	普通业务消息、任务通知

RocketMQ	事务消息和延迟消息能力较强	生态和使用成本相对较高	金融、电商、订单系统
线程池	实现简单，无需额外组件	可靠性和可恢复能力较弱	小规模后台任务

表 2-5　异步任务技术选型

本项目选择 Kafka 处理文档解析、OCR 识别、文本切片和向量化入库等异步任务。由于这些任务耗时较长，如果同步执行会影响用户体验，因此通过消息队列进行解耦，提高系统稳定性和可扩展性。

第三章 系统分析

3.1 系统业务需求

3.1.1 业务需求概述

本项目是一个面向企业知识管理场景的多模态 RAG 智能知识库系统，主要用于解决企业资料分散、检索不便、图片和扫描件资料难以利用、知识复用效率不高等问题。系统支持用户上传文档、图片、截图和扫描件等资料，通过文档解析、OCR 文字识别、文本切片、Embedding 向量化和混合检索等流程，将非结构化资料转化为可检索、可问答、可追溯的知识资源。

在使用过程中，用户可以通过自然语言与系统进行交互。系统会根据用户问题，从知识库中检索相关文档片段或 OCR 识别内容，并结合大语言模型生成回答。同时，系统提供引用来源展示、历史记录保存、权限控制、模型配置、模型选择、邀请码注册、套餐购买和微信支付等功能，满足企业知识管理、智能问答和系统运营管理等需求。

3.1.2 业务愿景

本系统的业务目标是构建一个集资料管理、OCR 识别、语义检索、智能问答、权限控制、模型管理和套餐支付于一体的智能知识服务平台。

系统主要业务愿景包括：

实现企业文档、图片、截图和扫描件等资料的统一管理。

通过 OCR 技术提取图片类资料中的文字内容，使其参与检索和问答。

通过 RAG 技术提升知识检索和问答回答的相关性。

通过关键词检索与语义检索结合，提高知识召回效果。

通过用户权限和组织标签机制，保障不同用户之间的数据隔离。

通过模型配置与模型选择机制，支持不同大语言模型和向量模型的接入。

通过邀请码系统控制用户注册入口，适用于内测、企业内部使用和限定范围开放场景。

通过套餐购买和微信支付功能，实现用户权益、额度和模型使用权限管理。

通过聊天记录和引用来源展示，提高知识问答过程的可追溯性。

3.1.3 涉众分析

涉众角色	主要需求
普通用户	上传个人资料，构建个人知识库，通过自然语言提问获取答案
企业用户	管理企业文档、技术资料、业务资料、截图和扫描件，提高知识复用效率
新员工或跨项目协作者	快速查询业务资料、操作说明、接口文档和常见问题
系统管理员	管理用户、组织、权限、套餐、订单、邀请码和模型配置
知识库维护人员	维护知识库资料，处理文档分类、标签、权限和更新
支付服务方	提供微信扫码支付能力，完成套餐购买流程
大语言模型服务	根据检索到的知识上下文生成回答
Embedding 模型服务	将文本内容转化为语义向量，用于后续检索
OCR 服务	对图片、截图和扫描件进行文字识别

表 3-1 涉众分析

3.1.4 业务过程分析

系统核心业务流程主要包括主业务流程、知识入库流程、智能问答流程、套餐购买流程、邀请码注册流程和模型配置选择流程。

主业务流程图如下：

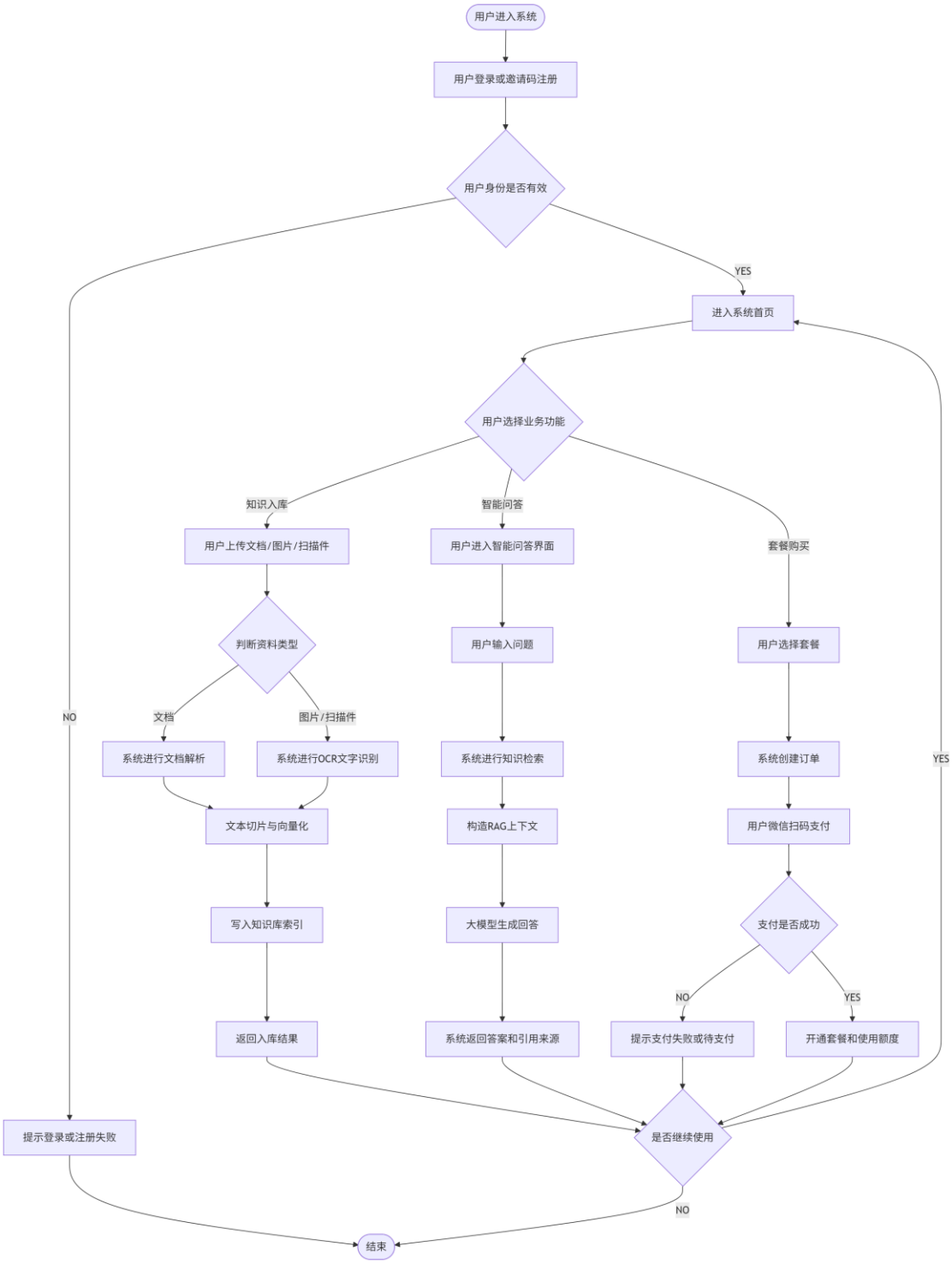

图 3-2 主业务流程图

1、知识入库流程

用户上传文档、图片或扫描件后，系统将原始文件存储到 MinIO 中，并根据文件类型进行处理。对于 Word、PDF、TXT 等文档，系统进行文本解析；对于图片、截图和扫描件，系统进行 OCR 文字识别。随后，系统将解析或识别出的文本进行切片，并调用 Embedding 模型生成向量表示，最终将文本片段、向

量数据和文件元数据存储到 Elasticsearch 和 MySQL 中。

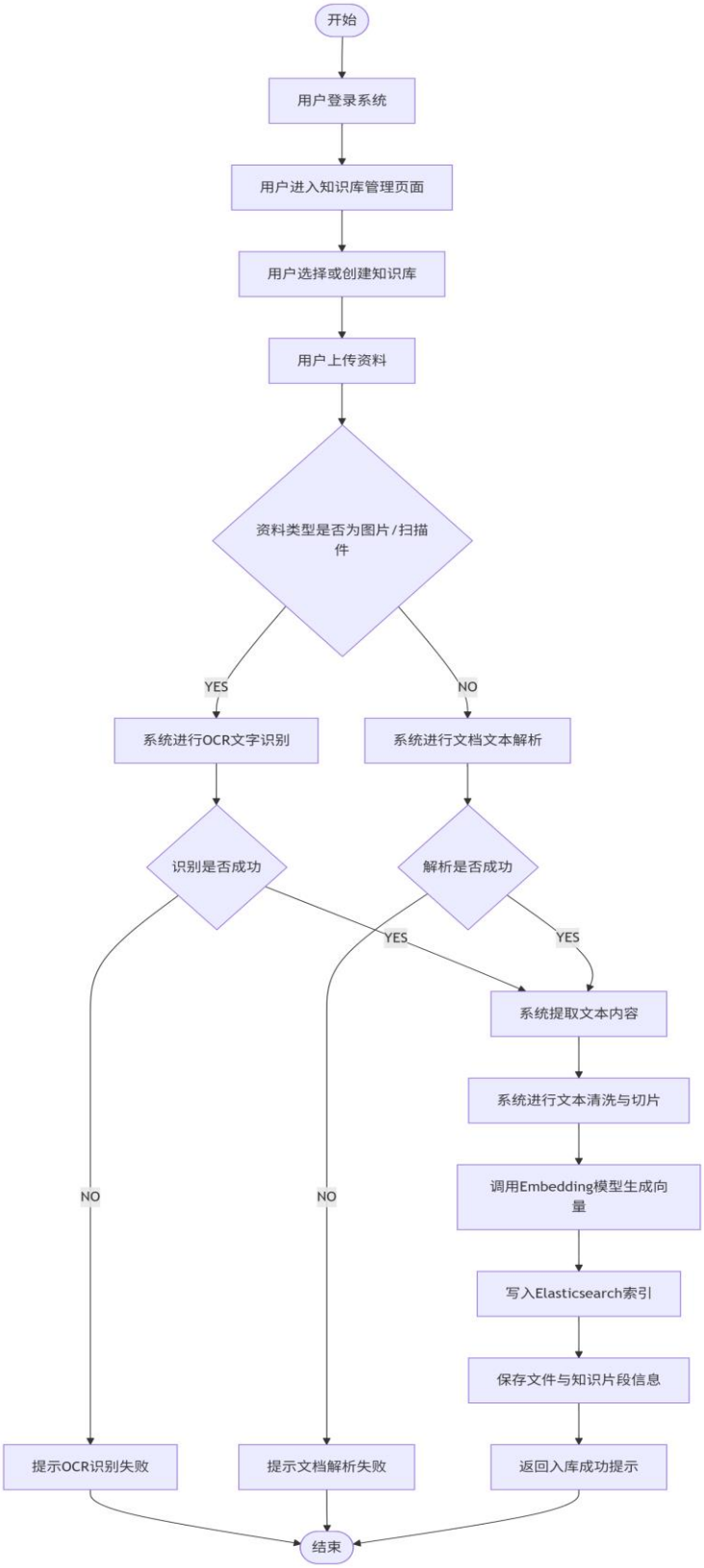

图 3-3 知识入库业务流程图

2、智能问答流程

用户在聊天页面输入问题后，系统先对问题进行向量化，然后在 Elasticsear

ch 中进行关键词检索和语义检索，召回相关知识片段。系统结合用户权限过滤不可访问内容，并将检索结果与用户问题共同发送给大语言模型。模型生成回答后，系统通过 WebSocket 流式返回给前端，同时保存对话记录和引用来源。

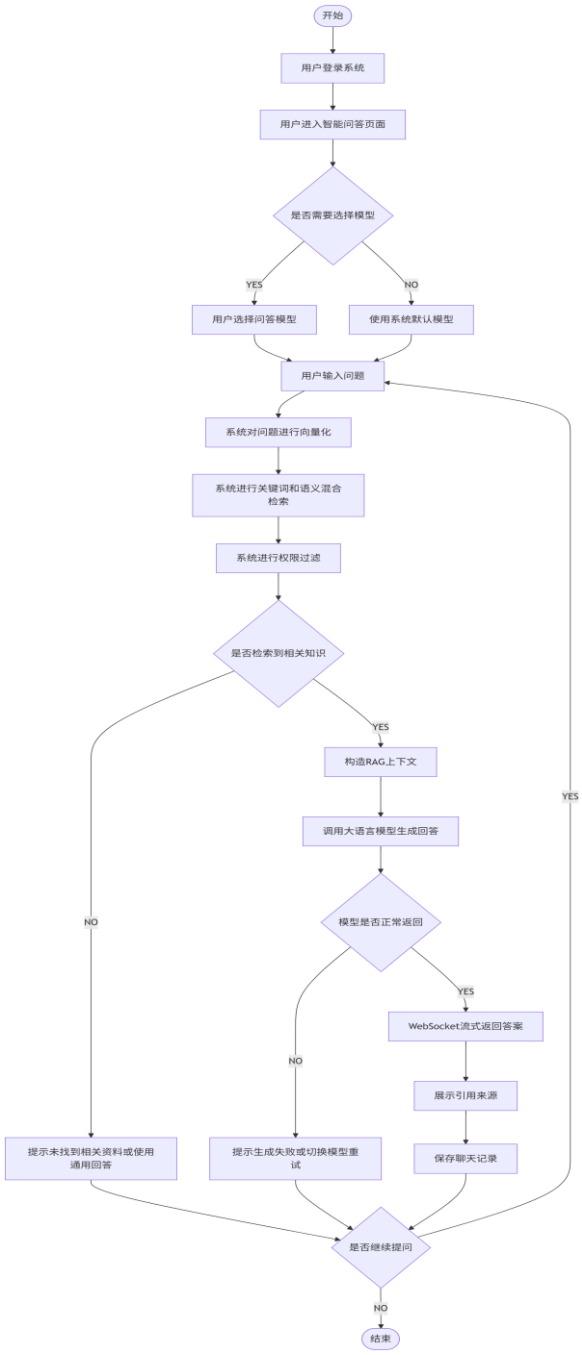

图 3-4　智能问答业务流程图

3、套餐购买流程

用户进入套餐购买页面后，选择对应套餐并创建订单。系统生成微信支付二维码，用户扫码完成支付后，系统确认支付结果，并更新订单状态，为用户开通相应套餐、额度或模型使用权限。

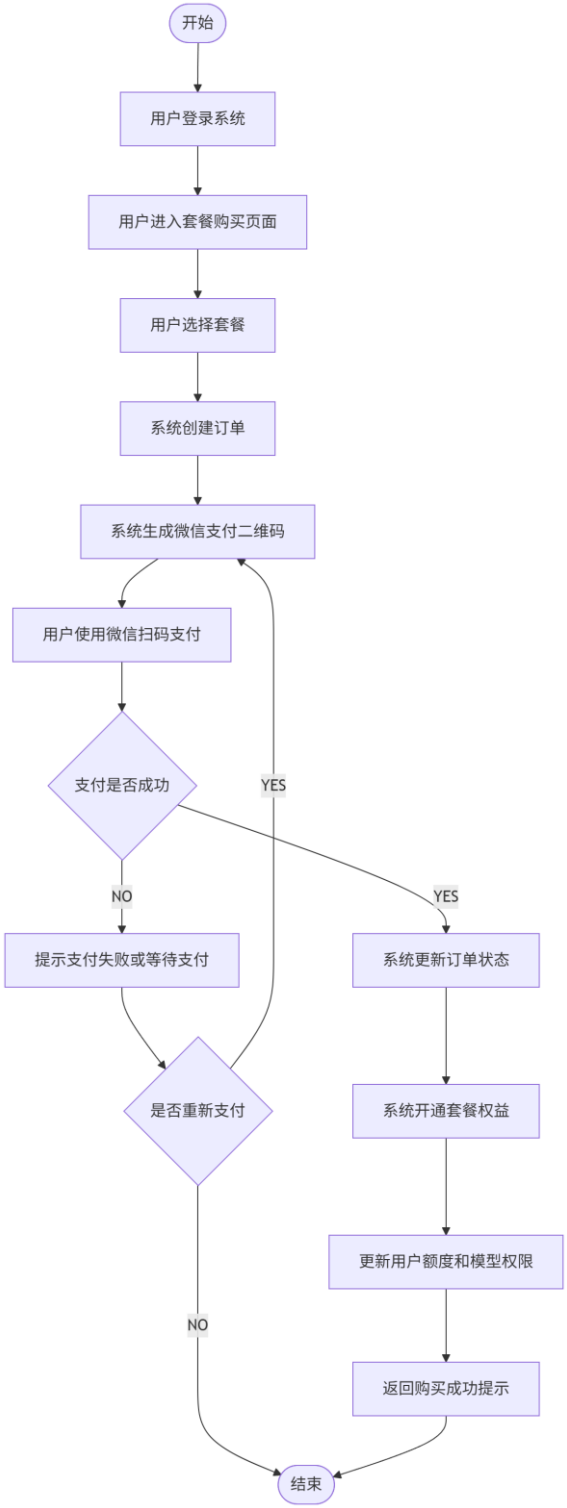

图 3-5　套餐购买业务流程图

4、邀请码注册流程

用户注册时需要填写邀请码。系统对邀请码的有效期、使用次数和启用状态进行校验，校验通过后创建用户账号，并绑定初始权限或套餐权益。

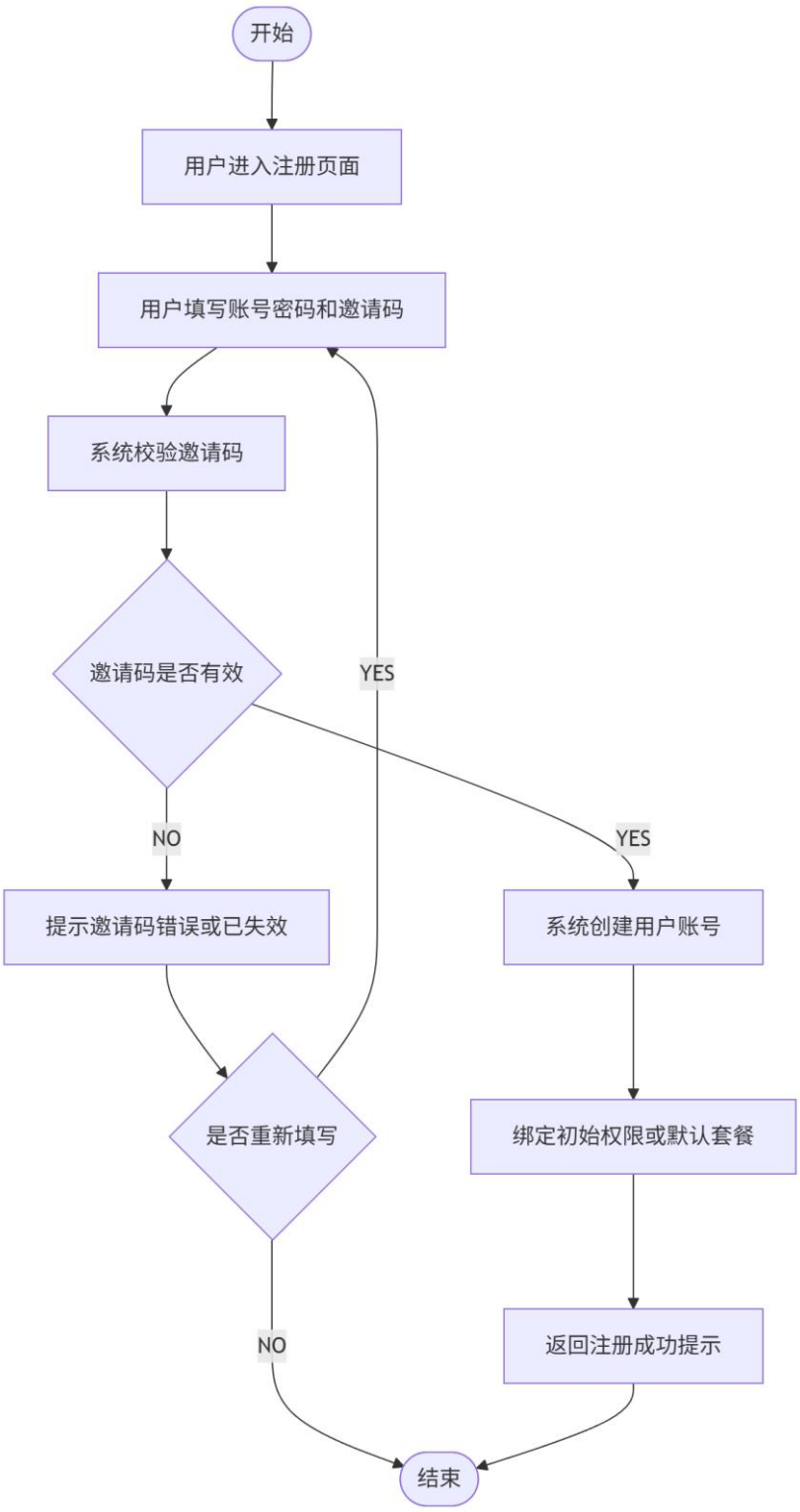

图 3-6 邀请码注册业务流程图

5、模型配置与选择流程

管理员可以在后台维护大语言模型和 Embedding 模型配置，包括模型名称、接口地址、密钥、模型类型和启用状态。用户或系统在问答、向量化等场景中根据权限和配置选择对应模型。

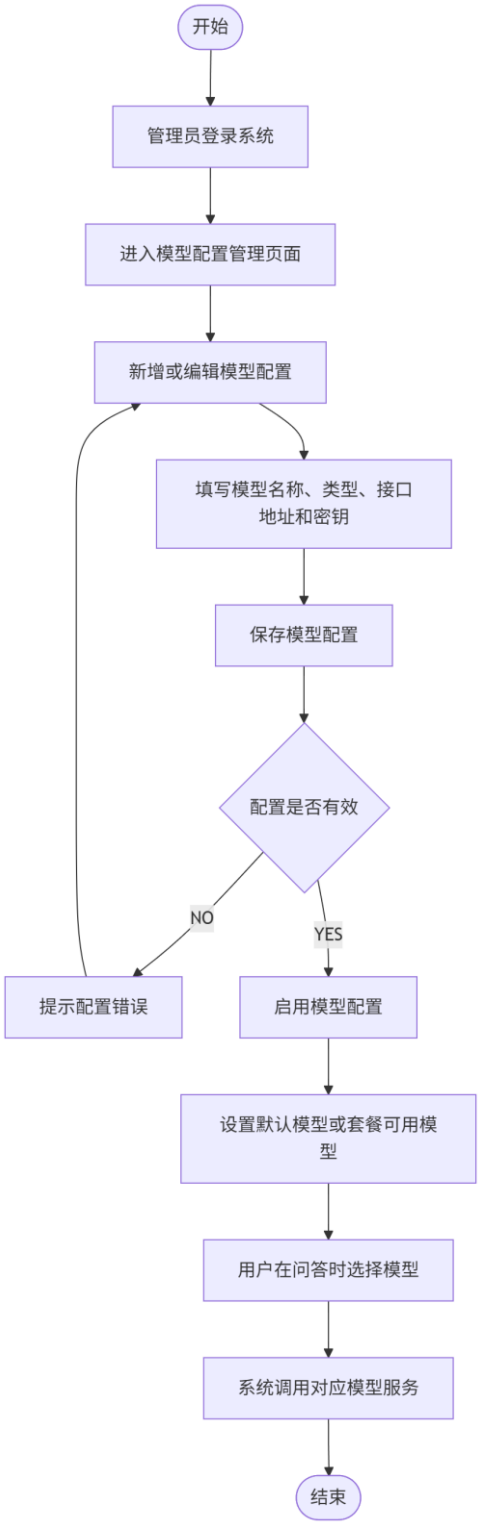

图 3-7 模型配置与选择业务流程图

3.2 系统功能需求

3.2.1 功能需求概述

本系统面向企业知识管理与智能问答场景，主要为用户提供文档、图片、截图和扫描件等资料的统一管理能力。系统通过文档解析、OCR 识别、文本切片、

向量化入库和混合检索等流程，将非结构化资料转化为可检索、可问答的知识资源。

在功能设计上，系统主要包括用户管理、知识库管理、文件上传与解析、OCR 识别、向量化与索引、知识检索、智能问答、聊天记录、套餐与微信支付、权限管理、模型配置与选择、邀请码管理和系统管理等模块。

系统主要参与者包括普通用户和系统管理员。普通用户主要使用知识库、上传资料、发起问答、查看历史记录、选择模型和购买套餐；系统管理员主要负责用户管理、邀请码管理、模型配置、套餐订单管理和系统基础配置维护。

3.2.2 系统顶层用例图

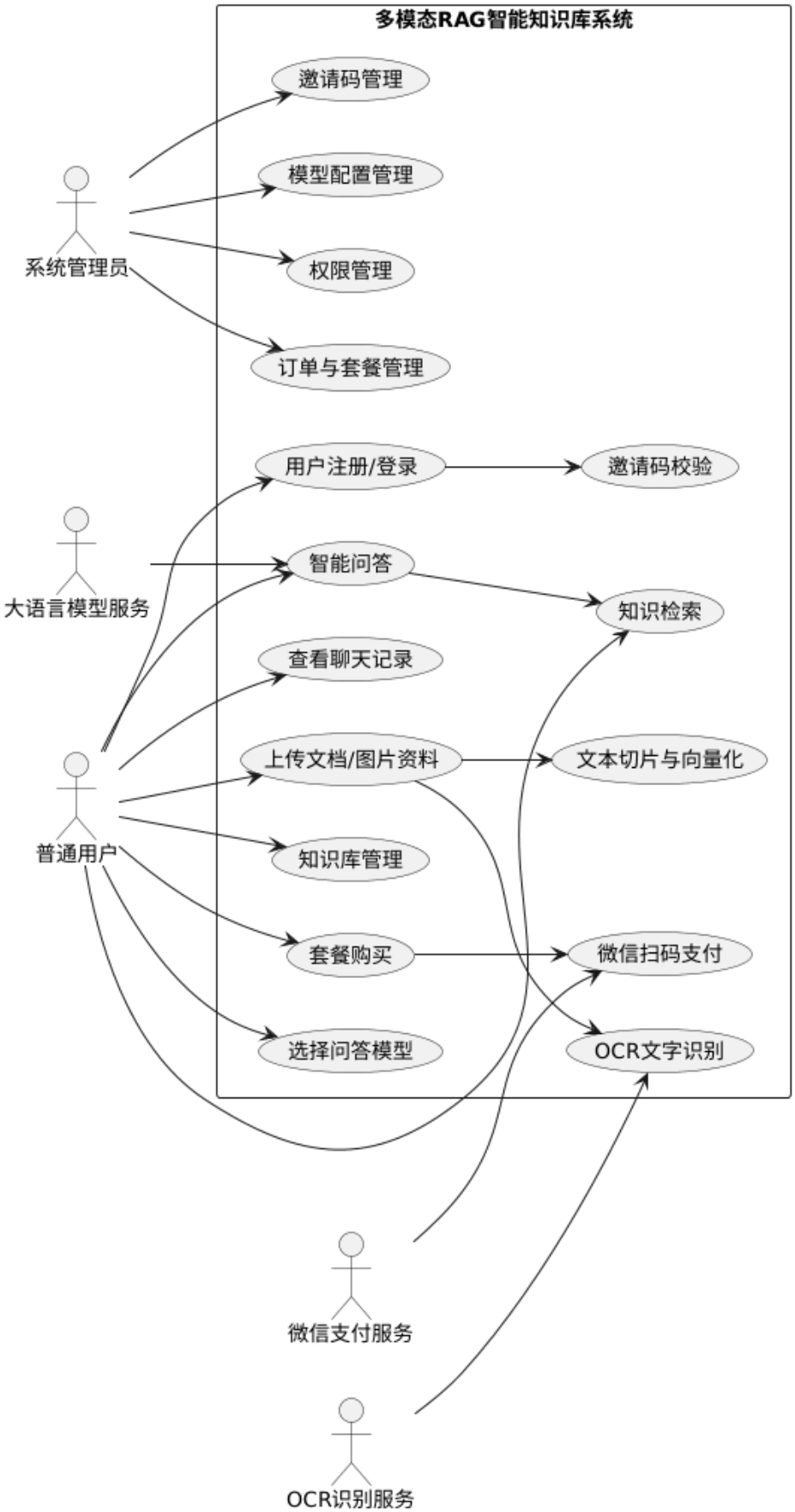
多模态RAG智能知识库系统
邀请码管理
模型配置管理
权限管理
订单与套餐管理
用户注册/登录
邀请码校验
智能问答
知识检索
查看聊天记录
上传文档/图片资料
文本切片与向量化
知识库管理
套餐购买
微信扫码支付
选择问答模型
OCR文字识别
系统管理员
大语言模型服务
普通用户
微信支付服务
OCR识别服务

图 3-8　系统顶层用例图

3.2.3 用户管理模块

用户管理模块主要负责用户注册、登录、身份认证和用户基础信息维护。系统通过邀请码控制注册入口，并通过 Token 机制识别用户身份。

功能名称	功能描述	参与者
用户注册	用户填写账号、密码和邀请码完成注册	普通用户
邀请码校验	系统校验邀请码是否有效、是否过期、是否超过使用次数	普通用户
用户登录	用户输入账号密码登录系统，系统返回身份认证令牌	普通用户
用户信息维护	用户查看或修改个人基础信息	普通用户
身份认证	系统根据 Token 判断用户身份是否合法	系统
角色识别	系统区分普通用户、管理员等不同角色	系统

表 3-9 用户管理模块表

3.2.4 知识库管理模块

知识库管理模块用于管理用户或组织下的知识空间。用户可以创建知识库，并将不同类型的资料归类到对应知识库中。

功能名称	功能描述	参与者
创建知识库	用户创建个人或组织知识库	普通用户
编辑知识库	修改知识库名称、描述、标签等信息	普通用户
删除知识库	删除不再使用的知识库及相关资源	普通用户
查看知识库列表	查看当前用户有权限访问的知识库	普通用户
设置知识库权限	设置知识库为个人、组织或公开范围	普通用户 / 管理员

表 3-10 知识库管理模块表

3.2.5 文件上传与解析模块

文件上传与解析模块是知识入库的起点。系统支持上传 Word、PDF、TXT 等文档，也支持上传图片、截图和扫描件。对于较大的文件，系统支持分片上传和断点续传。

功能名称	功能描述	参与者
文档上传	支持上传 Word、PDF、TXT 等文档	普通用户
图片上传	支持上传图片、截图、扫描件等资料	普通用户
分片上传	将大文件拆分为多个分片上传	普通用户
断点续传	上传中断后继续上传未完成部分	普通用户
文件存储	将原始文件保存到 MinIO 对象存储中	系统
文本解析	对文档进行文本抽取，为后续切片和向量化做准备	系统

表 3-11 文件上传与解析模块表

3.2.6 OCR 识别模块

OCR 识别模块用于处理图片、截图和扫描件等资料。系统通过 OCR 技术提取图像中的文字内容，并将识别结果纳入知识库处理流程。

功能名称	功能描述	参与者
图片文字识别	提取图片中的文字内容	系统
扫描件识别	识别扫描版文档中的文字信息	系统
截图内容识别	对报错截图、配置截图等进行文字抽取	系统
OCR 结果保存	保存识别出的文本内容和来源信息	系统
OCR 内容入库	将识别结果进行切片、向量化并写入索引	系统

表 3-12 OCR 识别模块表

3.2.7 向量化与索引模块

向量化与索引模块负责将文档解析结果和 OCR 识别结果转化为可检索的知识片段。系统先对文本进行切片，再调用 Embedding 模型生成向量，最后写入 Elasticsearch。

功能名称	功能描述	参与者
文本切片	将长文本拆分为适合检索的短文本片段	系统
Embedding 向量化	将文本片段转化为语义向量	系统
向量模型选择	系统根据管理员配置选择指定 Embedding 模型	系统
索引写入	将文本、向量和来源信息写入 Elasticsearch	系统
索引更新	文件更新后同步更新对应知识索引	系统
索引删除	文件删除后清理相关索引数据	系统

表 3-13 向量化与索引模块表

3.2.8 知识检索模块

知识检索模块负责根据用户问题召回相关知识内容。系统采用关键词检索与语义检索结合的方式，提高检索结果的相关性。

功能名称	功能描述	参与者
关键词检索	根据用户输入的关键词匹配相关文本	普通用户
语义检索	根据向量相似度查找语义相关内容	普通用户
混合检索	结合关键词检索和语义检索结果	系统
权限过滤	过滤用户无权访问的知识内容	系统
检索结果排序	根据相关性对检索结果进行排序	系统
来源信息返回	返回知识片段对应的文档、图片或扫描件来源	系统

表 3-14 知识检索模块表

3.2.9 智能问答模块

智能问答模块是系统的核心交互模块。用户可以通过自然语言提出问题，系统根据检索到的知识内容构造 RAG 上下文，并调用大语言模型生成回答。

功能名称	功能描述	参与者
自然语言提问	用户以对话方式输入问题	普通用户
问答模型选择	用户根据权限选择不同大语言模型进行问答	普通用户
RAG 上下文构造	将检索结果与用户问题组合为模型上下文	系统
大模型回答生成	调用大语言模型生成回答	系统
流式输出	通过 WebSocket 实时返回生成内容	系统
停止生成	用户可中途停止回答生成	普通用户
引用来源展示	展示回答依据来自哪些文档或 OCR 内容	系统

表 3-15 智能问答模块表

3.2.10 聊天记录模块

聊天记录模块用于保存用户与系统之间的问答内容，方便用户后续查看历史问题和回答。

功能名称	功能描述	参与者
新建会话	用户开始一次新的知识问答会话	普通用户
保存聊天记录	保存用户问题、模型回答和引用来源	系统
查看历史会话	用户查看过去的问答记录	普通用户
恢复上下文	系统根据历史记录恢复一定范围内的上下文	系统
删除会话	用户删除不需要的历史记录	普通用户

表 3-16 聊天记录模块表

3.2.11 套餐与微信支付模块

套餐与微信支付模块用于支持用户购买系统服务套餐。系统根据用户套餐控制其可使用的知识库容量、问答次数、Token 额度或模型权限。

功能名称	功能描述	参与者
套餐展示	展示不同套餐的价格、额度和权益	普通用户
创建订单	用户选择套餐后生成购买订单	普通用户
微信扫码支付	用户通过微信扫码完成支付	普通用户
支付状态确认	系统确认订单是否支付成功	系统
套餐开通	支付成功后开通对应套餐和额度	系统
订单记录查询	用户或管理员查看套餐购买记录	普通用户 / 管理员
额度扣减	用户使用问答或模型能力时扣减额度	系统
套餐模型权限	不同套餐可配置不同模型使用权限	管理员

表 3-17 套餐与微信支付模块表

3.2.12 权限管理模块

权限管理模块用于保障知识资源的访问安全。系统根据用户身份、角色和组织标签控制用户可以访问的文档、图片、OCR 内容和聊天记录。

功能名称	功能描述	参与者
角色权限控制	区分普通用户、管理员等角色权限	管理员
组织标签管理	根据组织标签划分用户和资料范围	管理员
私有知识控制	个人资料仅本人可访问	系统
组织知识共享	同组织用户可访问共享资料	系统
公开知识访问	公开资料允许所有用户访问	系统
OCR 内容权限继承	OCR 识别内容继承原始文件权限	系统

表 3-18 权限管理模块表

3.2.13 模型配置与选择模块

模型配置与选择模块用于管理系统接入的大语言模型和 Embedding 模型。管理员可以维护模型名称、接口地址、密钥、模型类型和启用状态，用户可根据权限选择对应模型进行问答。

功能名称	功能描述	参与者
新增模型配置	管理员新增大语言模型或 Embedding 模型配置	管理员
编辑模型配置	修改模型名称、接口地址、密钥、模型类型等信息	管理员
启用/停用模型	控制某个模型配置是否可被系统调用	管理员
默认模型设置	设置系统默认使用的大语言模型或向量模型	管理员
模型选择	用户或系统根据场景选择不同模型进行问答	普通用户 / 系统
模型调用记录	记录模型调用情况，便于统计和排查问题	系统
模型额度控制	根据套餐或权限限制用户可使用的模型能力	系统

表 3-19 模型配置与选择模块表

3.2.14 邀请码管理模块

邀请码管理模块用于控制用户注册入口。管理员可以创建邀请码，并设置邀请码的使用次数、有效期和启用状态。

功能名称	功能描述	参与者
创建邀请码	管理员生成新的邀请码	管理员
查看邀请码列表	查看邀请码内容、状态、使用次数和有效期	管理员
邀请码校验	用户注册时校验邀请码是否有效	系统
邀请码使用记录	记录邀请码被哪个用户使用	系统
邀请码失效	管理员手动禁用邀请码或到期自动失效	管理员
注册准入控制	未提供有效邀请码的用户无法完成注册	系统

表 3-20 邀请码管理模块表

3.2.15 系统管理模块

系统管理模块主要面向管理员，用于维护系统运行所需的基础配置和业务数据。

功能名称	功能描述	参与者
用户管理	管理用户账号、状态和角色	管理员
文件管理	查看和维护系统中的上传文件	管理员
知识库管理	管理知识库分类、状态和权限	管理员
订单管理	查看套餐订单和支付状态	管理员
模型配置管理	管理大语言模型和 Embedding 模型参数	管理员
邀请码管理	管理邀请码创建、启用、停用和使用记录	管理员
系统日志查看	查看文件处理、OCR、问答和支付相关日志	管理员

表 3-21 系统管理模块表

3.2.16 关键用例说明

用例名称	参与者	前置条件	基本流程	后置结果
使用邀请码注册	普通用户	用户获得有效邀请码	用户填写注册信息和邀请码，系统校验邀请码并创建账号	用户注册成功并获得初始权限
上传文档资料	普通用户	用户已登录	用户选择知识库并上传文档，系统保存文件并解析文本	文档进入知识库处理流程
上传图片并 OCR 识别	普通用户	用户已登录	用户上传图片或扫描件，系统进行 OCR 识别并保存文本	图片内容转化为可检索知识
智能问答	普通用户	用户已登录且知识库已有内容	用户输入问题，系统检索知识片段并调用大模型生成回答	用户获得回答和引用来源
选择问答模型	普通用户	用户具备模型使用权限	用户选择模型并发起提问，系统调用对应模型生成回答	用户获得指定模型生成的回答
查看聊天历史	普通用户	用户已有问答记录	用户进入历史记录页面，查看历史会话和回答	历史问答内容被展示
购买套餐	普通用户	用户已登录	用户选择套餐，生成订单并通过微信扫码支付	支付成功后套餐权益生效
配置模型服务	管理员	管理员已登录	管理员填写模型名称、接口地址、密钥和模型类型并保存	系统新增可用模型配置
管理邀请码	管理员	管理员已登录	管理员创建、启用、停用邀请码并查看使用记录	邀请码状态被更新
管理用户权限	管理员	管理员已登录	管理员设置用户角色、组织标签和访问范围	用户权限被更新

表 3-22 关键用例说明表

3.2.17 关键活动图

用户上传资料并入库活动图：

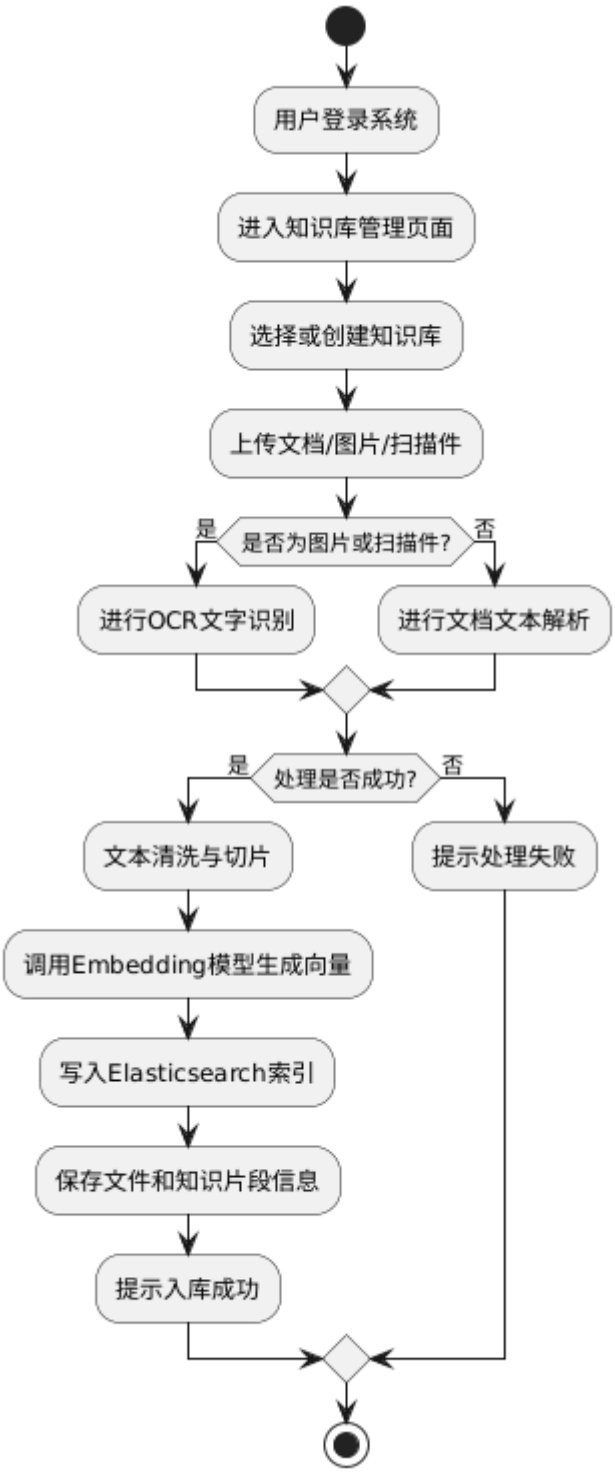

图 3-23 用户上传资料并入库活动图

智能问答活动图：

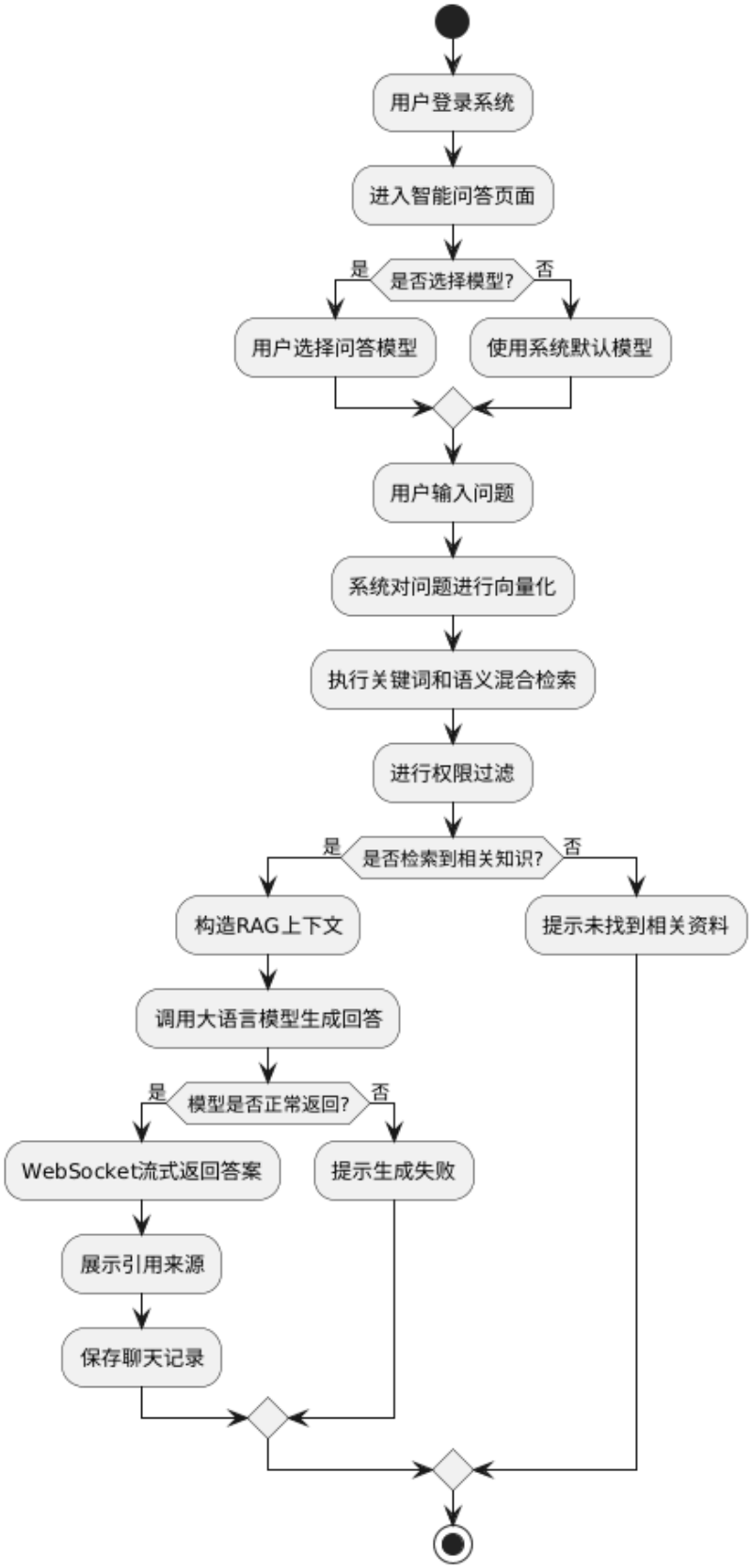

图 3-24　智能问答活动图

套餐购买活动图：

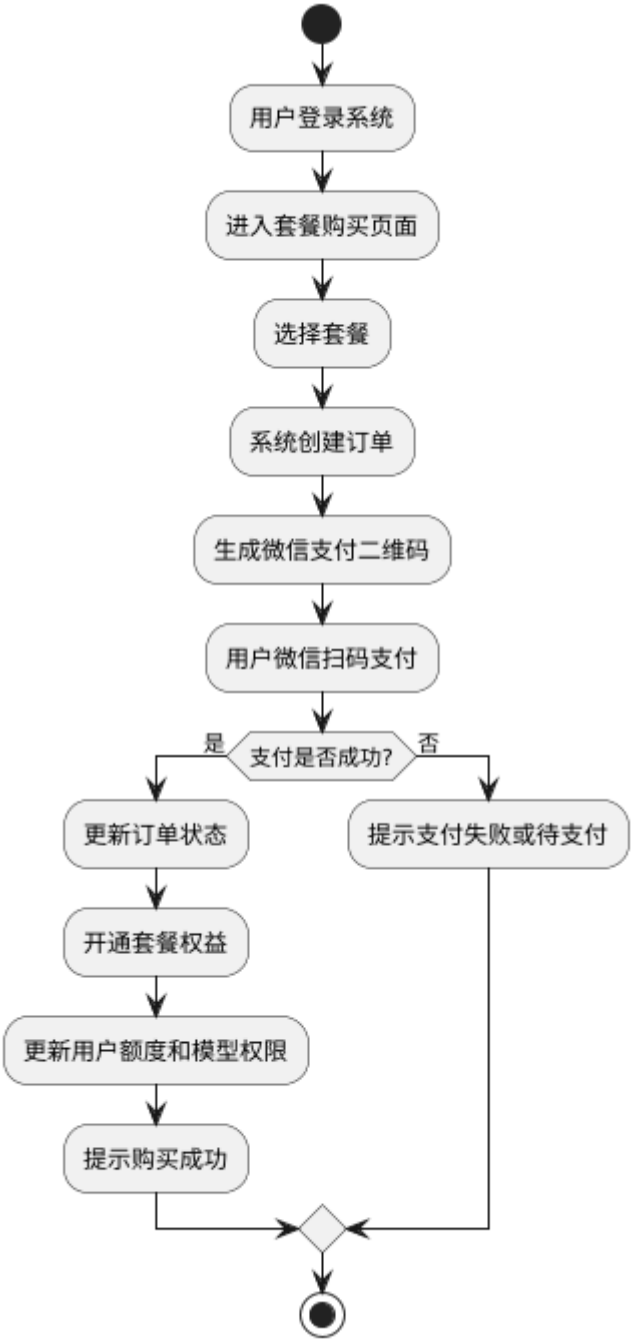

图 3-25　套餐购买活动图

邀请码注册活动图：

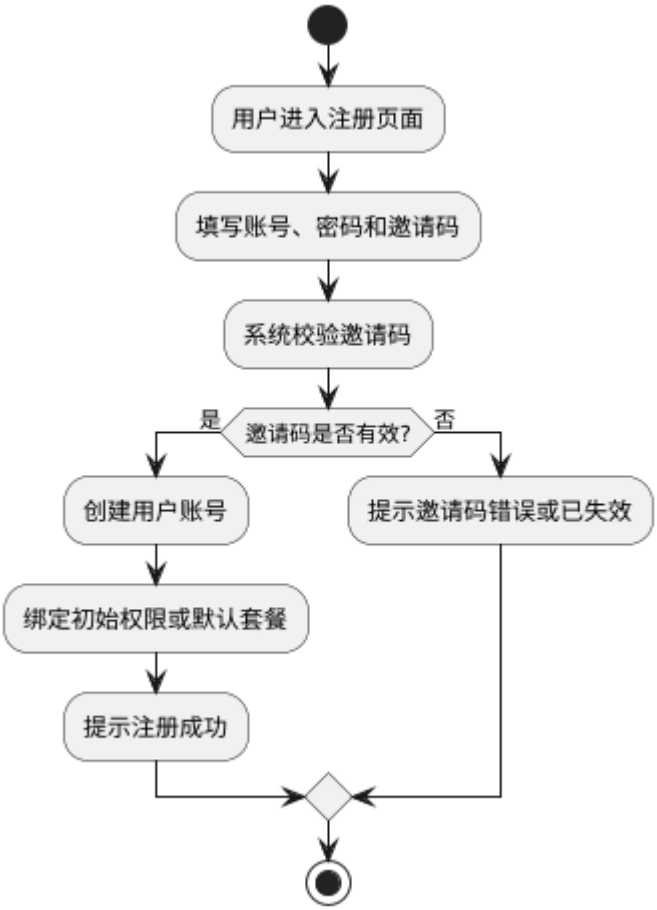

图 3-26　邀请码注册活动图

模型配置与选择活动图：

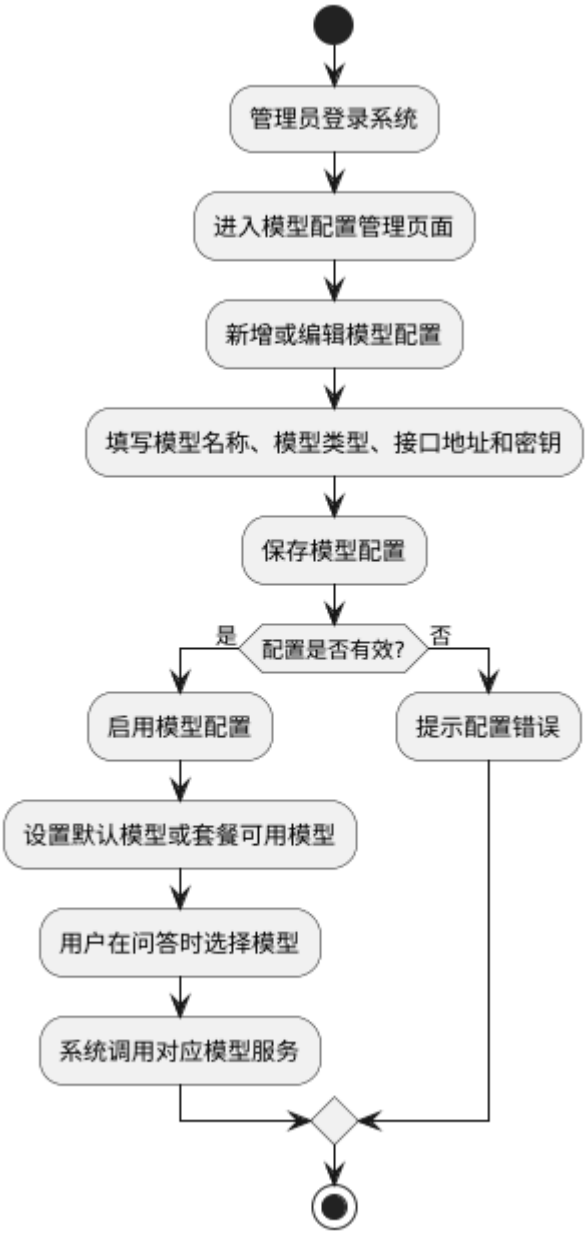

图 3-27 模型配置与选择活动图

3.3 系统非功能性需求

3.3.1 性能需求

系统需要满足日常知识库管理和智能问答的基本性能要求。在普通网络环境下，用户登录、知识库列表查询、聊天记录查询等常规接口应具有较快响应速度。对于智能问答场景，系统采用 WebSocket 流式输出方式，使用户能够尽快看到模型生成内容，减少等待感。

对于文档解析、OCR 识别和向量化等耗时任务，系统通过 Kafka 进行异步

处理，避免长时间阻塞用户请求。文件上传过程支持分片上传和断点续传，以提高大文件上传的稳定性。

3.3.2 易用性需求

系统界面应保持清晰、简洁，用户可以较容易地完成注册登录、上传资料、创建知识库、发起问答、查看历史记录和购买套餐等操作。智能问答功能应支持自然语言输入，用户无需学习复杂指令即可使用系统。

对于文档、图片和扫描件上传，系统应提供明确的状态提示，例如上传中、解析中、OCR 识别中、向量化中、处理完成或处理失败等，方便用户了解任务进度。

3.3.3 安全性需求

系统需要对用户身份进行认证，并对不同用户、角色和组织的数据访问进行权限控制。用户只能访问自己有权限的知识库、文档、OCR 识别内容和聊天记录。

系统中的 API Key、模型密钥、支付相关配置等敏感信息应避免明文暴露。套餐购买和支付流程需要记录订单状态，防止重复开通或异常开通。OCR 识别后的文本内容应继承原始文件权限，避免图片或扫描件转化为文本后造成权限绕过。

3.3.4 可靠性需求

系统应具备一定的异常处理能力。当文件上传中断时，用户可以继续上传未完成部分；当文档解析、OCR 识别或向量化失败时，系统应记录失败原因，并允许用户重新处理或重新上传。

对于智能问答过程，系统应保存必要的对话状态和历史记录。当网络波动或页面刷新时，用户仍可查看已保存的历史对话。对于异步任务，系统应记录任务状态，便于后续排查问题。

3.3.5 可扩展性需求

系统应具备良好的扩展能力。模型配置模块应支持后续接入更多大语言模型和 Embedding 模型；OCR 模块应便于替换不同 OCR 服务；知识检索模块应支持后续优化检索排序策略；套餐模块应支持新增套餐类型、调整额度规则和模型权限。

系统采用前后端分离结构，后端业务模块之间保持相对独立，有利于后续扩展新功能。

3.3.6 可维护性需求

系统代码结构应清晰，前后端模块划分明确。后端按照控制层、服务层、数据访问层等方式组织代码，前端按照页面、组件、接口服务和状态管理等方式组织代码。系统应记录关键业务日志，包括用户登录、文件上传、OCR 识别、模型调用、套餐购买和支付状态等，便于问题定位和维护。

3.3.7 运行环境需求

系统后端运行环境主要包括 Java、Spring Boot、MySQL、Redis、Elasticsearch、Kafka 和 MinIO 等服务；前端运行环境主要包括 Vue 3、TypeScript、Vite 和现代浏览器。系统需要能够在普通服务器或本地开发环境中运行，并支持根据实际部署需求进行配置调整。

3.4 可行性分析

3.4.1 技术可行性

本项目所采用的 Spring Boot、Vue 3、MySQL、Redis、Elasticsearch、Kafka 和 MinIO 等技术均较为成熟，具备较完整的社区生态和工程实践基础。OCR、Embedding 向量化和大语言模型接口也已经具备较多可用方案，能够满足图片文字识别、语义检索和智能问答等功能需求。

检索增强生成（RAG）框架通过将预测结果与外部证据相结合来增强逻辑逻辑模型（LLM），从而提高可靠性和事实性[6]。系统采用 RAG 架构，将知识检索与大语言模型生成结合，整体技术路线清晰。通过 Elasticsearch 实现关键词检索与向量检索，通过 Kafka 处理耗时任务，通过 MinIO 存储原始文件，通过 Redis 缓存短期状态，技术组合具备可实现性。

3.4.2 操作可行性

系统面向普通用户和企业用户，主要操作包括注册登录、上传资料、创建知识库、发起问答、查看历史记录和购买套餐等，符合常见 Web 系统的使用习惯。用户可以通过自然语言提问，无需掌握复杂检索语法。

管理员可以通过后台管理用户、邀请码、模型配置、套餐和订单等信息，操作路径相对清晰。因此，从使用和管理角度看，系统具有较好的操作可行性。

3.4.3 经济可行性

本项目主要使用开源框架和常见基础设施，开发成本相对可控。Spring Boot、Vue 3、MySQL、Redis、Elasticsearch、Kafka 和 MinIO 均可在本地或普通服务器上部署。OCR 和大语言模型能力可根据实际情况选择本地模型或第三方 API 服务。

系统通过套餐购买和微信支付模块支持用户权益管理，为后续服务运营提供了基础。因此，从开发成本和后续使用价值角度看，项目具有一定经济可行性。

3.4.4 法律与合规可行性

系统主要处理用户主动上传的文档、图片和扫描件，因此需要在使用过程中提醒用户遵守相关法律法规，不上传侵犯他人权益或包含非法内容的资料。对于企业内部资料，系统通过用户认证、权限控制和组织标签机制降低越权访问风险。

支付功能应记录订单和支付状态，并根据实际应用环境合理配置支付方式。模型 API Key、用户数据、文件资料和 OCR 识别内容应妥善保存，避免敏感信息泄露。整体来看，在合理权限控制和数据管理前提下，项目具备法律与合规可行性。

3.4.5 社会可行性

随着企业数字化和人工智能应用的发展，企业对知识管理、智能检索和自动问答的需求不断增加。传统文档管理方式难以充分利用大量非结构化资料，尤其是图片、截图和扫描件等内容。本系统通过 OCR、多模态资料处理和 RAG 技术，提高知识资源的利用效率，具有一定的实际应用价值。

同时，系统也适用于个人学习、企业培训、技术支持和内部知识共享等场景，能够帮助用户减少重复查找资料的时间，提高知识获取效率。因此，本项目具有较好的社会可行性。

第四章 系统设计

4.1 系统总体设计

4.1.1 系统设计目标

本系统面向企业知识管理和智能问答场景，设计目标是构建一个支持多模态资料处理的 RAG 智能知识库平台。系统通过文档解析、OCR 文字识别、文本切片、向量化、混合检索和大语言模型生成等技术，将企业内部的文档、图片、截图和扫描件等非结构化资料转化为可检索、可问答、可追溯的知识资源。

系统不仅支持基础的知识库构建和智能问答，还提供用户权限管理、组织标签隔离、模型配置与模型选择、邀请码注册、套餐购买和微信支付等功能，使系统具备较完整的用户管理、知识管理和运营管理能力。

系统总体设计目标如下：

设计目标	说明
知识统一管理	支持文档、图片、截图、扫描件等资料统一上传、存储和管理
多模态资料处理	通过 OCR 识别图片和扫描件中的文字内容，使其参与知识检索
智能检索问答	基于 RAG 技术实现知识检索与大语言模型问答结合
权限安全控制	通过用户角色、组织标签和资源可见性实现数据访问隔离
模型灵活配置	支持管理员配置不同大语言模型和 Embedding 模型
用户自主选择模型	支持用户根据权限或套餐选择可用模型进行问答
用户准入控制	通过邀请码系统控制注册入口，适用于内测或企业内部使用
套餐支付管理	支持套餐购买、微信扫码支付、额度管理和模型权限控制

表 4-1 系统总体设计目标表

4.1.2 系统总体架构

本系统采用前后端分离的分层架构设计，整体可以划分为用户访问层、前端展示层、接口与安全层、业务逻辑层、数据访问层、数据存储层和外部服务层。

用户通过浏览器访问前端页面，前端通过 HTTP 接口和 WebSocket 与后端进行交互。后端基于 Spring Boot 提供统一的业务接口，并通过 Spring Security 和 JWT 完成身份认证与权限控制。系统业务逻辑层负责用户管理、知识库管理、文件处理、OCR 识别、模型配置、RAG 问答、支付订单等核心业务。数据层则由 MySQL、Redis、Elasticsearch 和 MinIO 共同组成，分别承担结构化数据存储、缓存、检索索引和文件存储等职责。该架构结合了用于高效证据选择的聚类密集检索、用于多跳推理的基于图的依赖关系遍历、用于组合推理的递归检索

-生成循环以及用于降低端到端延迟的缓存层[8]。

系统总体架构如下：

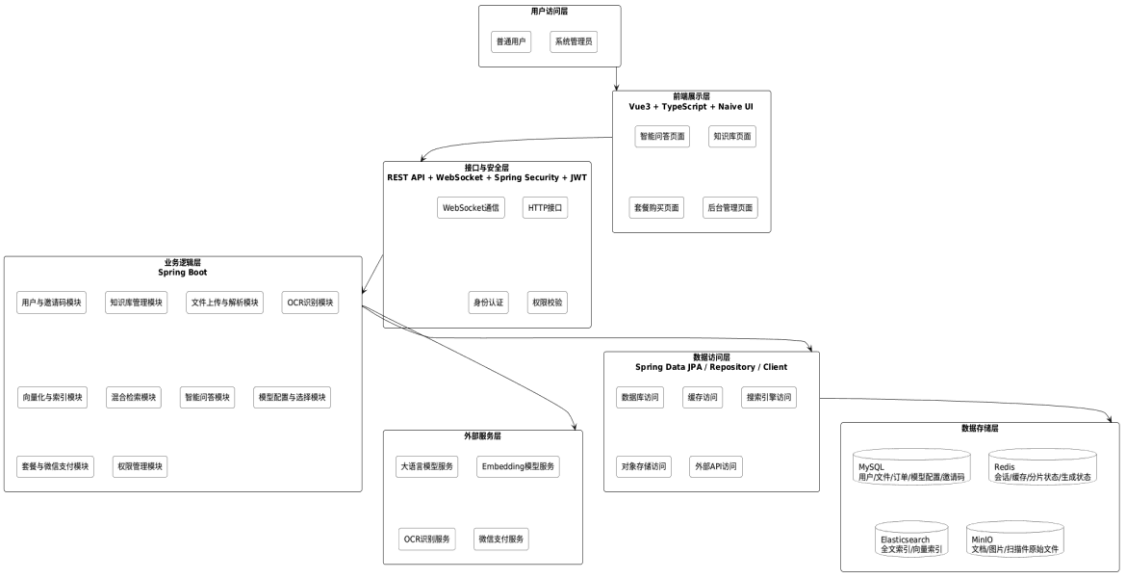

图 4-2　系统总体架构图

4.1.3 业务架构设计

从业务角度看，系统可以划分为用户层、业务逻辑层和数据资源层。

用户层是系统入口，主要包括普通用户和系统管理员。普通用户可以进行注册登录、上传资料、创建知识库、发起智能问答、查看历史记录、选择模型和购买套餐等操作。系统管理员主要负责用户管理、邀请码管理、组织权限管理、模型配置、套餐订单管理和系统运行维护。

业务逻辑层是系统核心，主要包括用户管理、知识库管理、文件处理、OCR 识别、知识检索、智能问答、模型管理、套餐支付和权限控制等模块。各模块之间通过统一接口协作，共同完成从资料入库到智能问答的完整流程。

数据资源层负责保存系统运行过程中产生的各类数据，包括用户信息、组织标签、邀请码、套餐订单、文件元数据、OCR 识别结果、文本切片、向量索引、聊天记录和原始文件等。

架构层次	组成内容	主要职责
用户层	普通用户、系统管理员	使用系统功能、维护系统配置
业务逻辑层	用户管理、知识库管理、OCR、检索、问答、支付、模型管理等	处理核心业务逻辑
数据资源层	MySQL、Redis、Elasticsearch、MinIO	存储业务数据、缓存状态、索引数据和原始文件
外部服务层	大语言模型、Embedding、OCR、微信支付	提供模型生成、向量化、文字识别和支付能力

表 4-3 业务架构设计表

4.1.4 功能模块划分

根据系统业务需求，系统主要划分为以下功能模块。

模块名称	主要功能
用户管理模块	用户注册、登录、身份认证、用户信息维护
邀请码管理模块	邀请码生成、校验、启用、停用和使用记录管理
知识库管理模块	知识库创建、编辑、删除、列表查询和权限设置
文件上传与解析模块	文档上传、图片上传、分片上传、断点续传和文本解析
OCR 识别模块	图片、截图、扫描件文字识别和识别结果入库
向量化与索引模块	文本切片、Embedding 向量化、索引写入、索引更新和删除
知识检索模块	关键词检索、语义检索、混合检索、权限过滤和结果排序
智能问答模块	RAG 上下文构造、大模型回答生成、WebSocket 流式输出
聊天记录模块	会话保存、历史记录查询、上下文恢复和记录删除
权限管理模块	角色权限、组织标签、私有知识、组织知识和公开知识控制
模型配置与选择模块	大语言模型和 Embedding 模型配置、启用、停用、默认模型设置和模型选择
套餐与微信支付模块	套餐展示、订单创建、微信扫码支付、套餐开通和额度扣减
系统管理模块	用户、文件、订单、模型、邀请码和系统日志管理

表 4-4 功能模块划分表

其中，模型配置与选择模块是系统灵活接入不同模型服务的重要支撑。管理员可以在后台配置模型名称、模型类型、接口地址、API Key、启用状态等信息；用户在问答时可以根据权限或套餐选择不同模型。邀请码管理模块用于控制系统注册入口，适合企业内部使用、测试开放和限定范围注册等场景。套餐与微信支付模块则用于实现服务权益管理，使系统能够根据用户购买情况控制问答次数、Token 额度、模型权限等资源。

4.1.5 核心业务流程设计

1、多模态知识入库流程

多模态知识入库是系统的基础流程。用户上传资料后，系统首先保存原始文件，并根据文件类型选择处理方式。对于 Word、PDF、TXT 等文档，系统调用文档解析服务提取文本；首先，利用命名实体识别和关系抽取构建文档图。然后，对所有文档图进行剪枝以减少冗余，接着聚合文档图对之间的层级关系特征[16]。对于图片、截图和扫描件，系统调用 OCR 服务识别文字。随后系统对文本进行清洗、切片和向量化处理，并将结果写入 Elasticsearch，供后续检索使用。

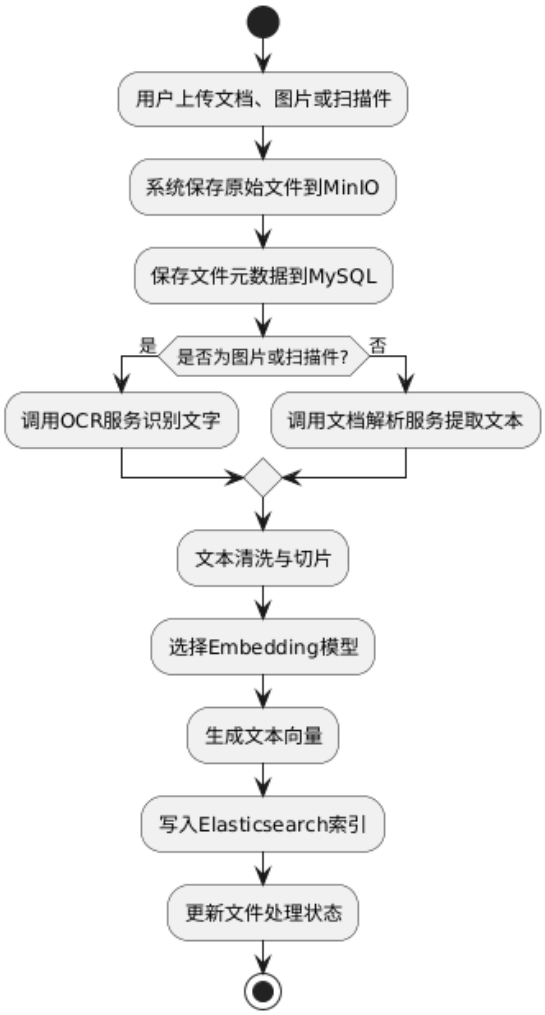

图 4-5 多模态知识入库流程图

2、智能问答流程

智能问答流程是系统的核心流程。用户输入问题后，系统根据用户选择的模型和权限信息，对问题进行向量化处理，并在 Elasticsearch 中执行混合检索。检索结果经过权限过滤后，与用户问题和历史上下文共同构造成 Prompt，发送给大语言模型。模型生成的回答通过 WebSocket 流式返回给前端，并保存聊天记录和引用来源。

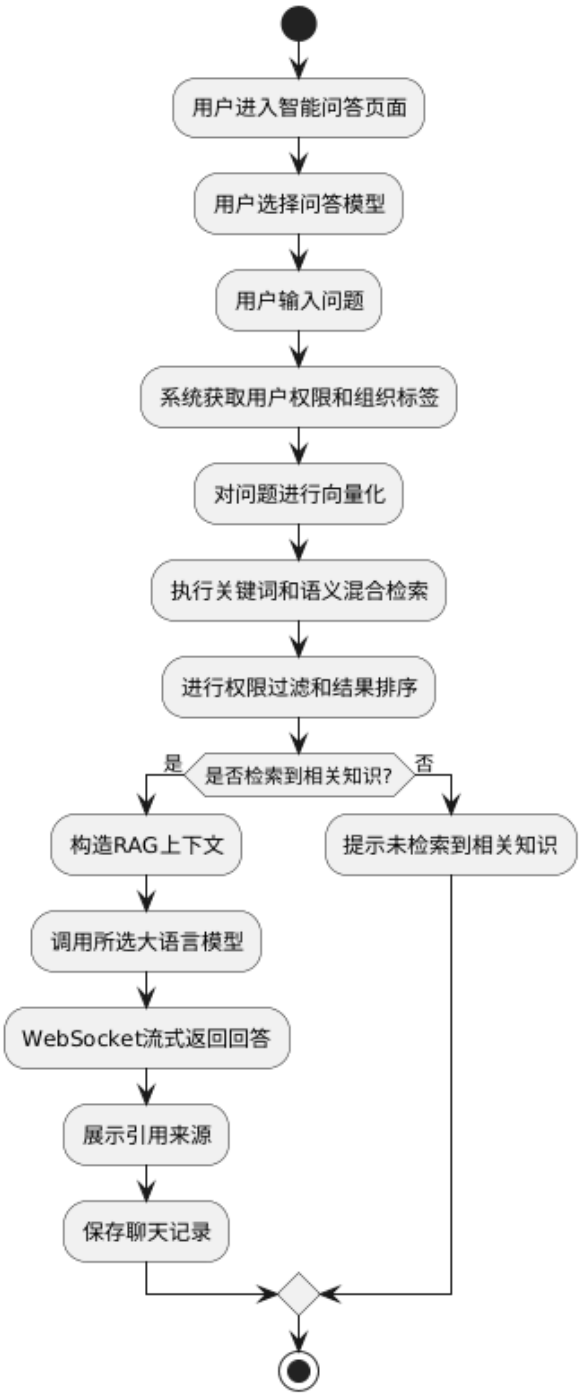

图 4-6 智能问答流程图

3、模型配置与选择流程

模型配置与选择流程用于提升系统对不同模型服务的适配能力。管理员可以维护多个大语言模型和向量模型配置，系统根据默认配置、用户选择或套餐权限决定实际调用的模型。

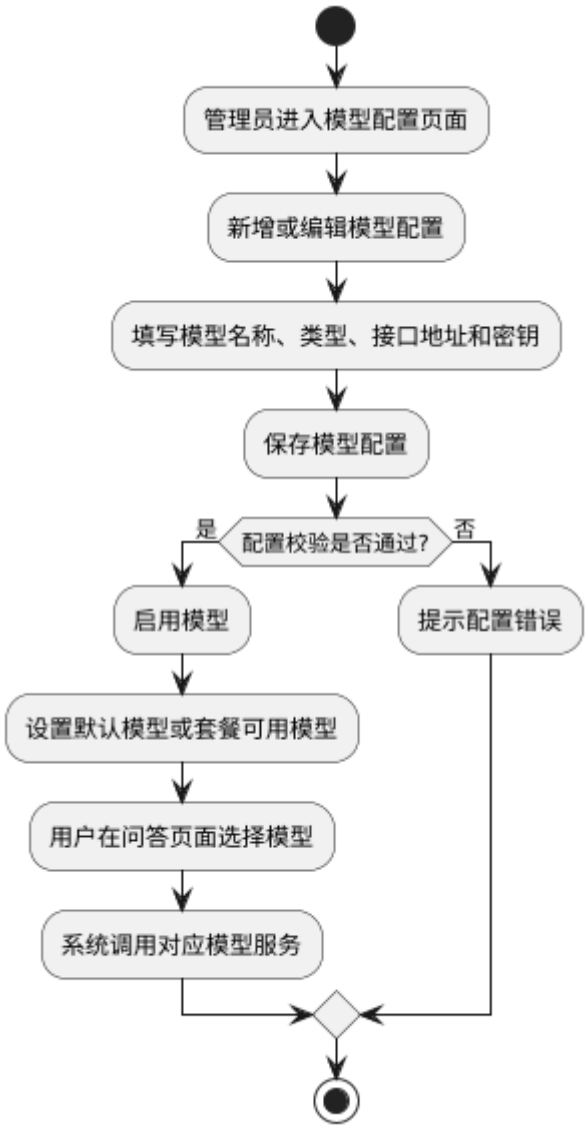

图 4-7 模型配置与选择流程图

4、邀请码注册流程

邀请码注册流程用于控制用户准入。用户注册时需要填写有效邀请码，系统校验邀请码状态、有效期和使用次数，校验通过后才允许创建账号。

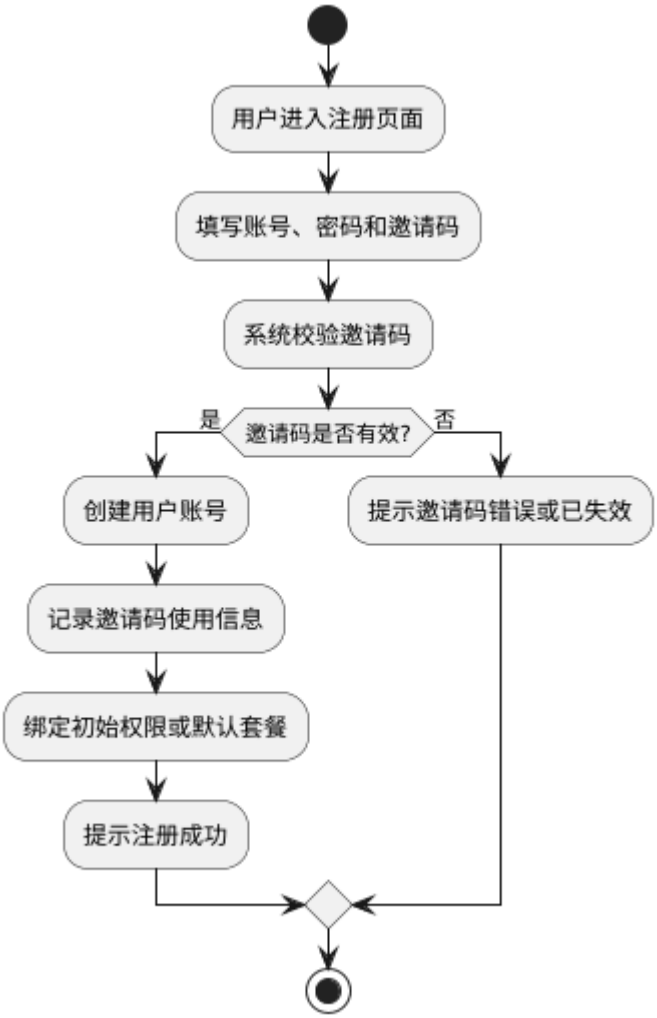

图 4-8 邀请码注册流程图

5、套餐购买与微信支付流程

套餐购买流程用于管理用户权益。用户选择套餐后，系统创建订单并生成微信支付二维码。用户扫码支付后，系统确认支付结果，并更新订单状态、套餐权益、问答额度和可用模型范围。

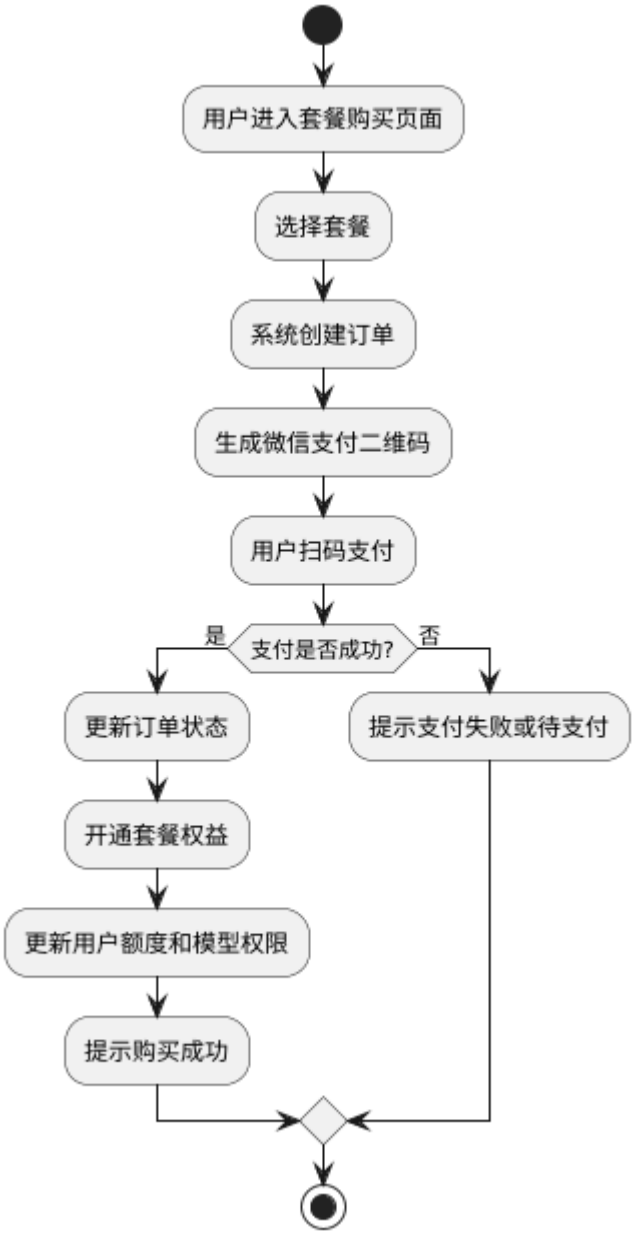

图 4-9 套餐购买与微信支付流程图

4.1.6 数据流转设计

系统的数据流转主要围绕“资料入库”和“智能问答”两条主线展开。

在资料入库过程中，用户上传的原始文件进入 MinIO，文件元数据进入 MySQL。文档解析或 OCR 识别后得到的文本内容会经过切片和向量化处理，最终写入 Elasticsearch。处理状态、文件信息、权限信息和来源信息同步保存在 MySQL 中。

在智能问答过程中，用户问题先经过向量化，再进入 Elasticsearch 进行混合检索。系统根据用户权限过滤结果后，将相关知识片段与问题组合成 Prompt，

发送给大语言模型。模型生成的回答返回给前端，同时保存到 MySQL 或 Redis 中，用于历史记录和上下文管理。

数据类型	存储位置	用途
用户信息	MySQL	登录认证、用户管理
邀请码信息	MySQL	注册准入控制
模型配置	MySQL	模型调用、模型选择、套餐权限
套餐订单	MySQL	支付记录、套餐权益管理
原始文件	MinIO	保存文档、图片、扫描件
文件元数据	MySQL	文件管理、权限控制、状态跟踪
OCR 识别文本	MySQL / Elasticsearch	文字保存、检索和问答
文本向量	Elasticsearch	语义检索
对话上下文	Redis	短期上下文缓存
聊天记录	MySQL	历史记录查询和追溯
分片上传状态	Redis	大文件断点续传
权限缓存	Redis	提高权限判断效率

表 4-10 数据流转设计表

4.1.7 系统设计要点

1、多模态知识处理设计：近年来，大型语言模型（LLM）在知识组织、自然语言理解和上下文推理方面展现出卓越的能力。然而，LLM 在应用于知识密集型领域时仍存在显著的局限性[7]。本系统不仅处理传统文档，还支持图片、截图和扫描件等资料。对于图片类资料，系统通过 OCR 提取文字内容，再统一进入文本切片、向量化和索引流程。这样可以使原本难以检索的图片资料转化为可检索知识，提升知识库覆盖范围。

2、RAG 问答设计：系统采用“先检索、后生成”的 RAG 架构。用户提问后，系统先从知识库中召回相关内容，再将检索结果作为上下文交给大语言模型生成回答。该设计能够使回答更贴近用户上传的知识资料，并通过引用来源提升结果可追溯性。

3、混合检索设计：系统将关键词检索和语义检索结合起来。关键词检索适合查找接口名、配置项、专业术语等精确内容；语义检索适合理解用户自然语言表达。两者结合能够提高检索结果的覆盖率和相关性。

4、权限隔离设计：系统通过角色权限、组织标签和资源可见性实现访问控

制。文档、图片、OCR 识别结果和向量索引均继承原始资源的权限信息，用户只能检索和查看自己有权限访问的知识内容。

5、模型配置与选择设计：系统支持管理员配置多个大语言模型和 Embedding 模型，并可根据实际需求启用、停用或设置默认模型。用户在问答时可以根据权限或套餐选择可用模型。该设计提高了系统对不同模型服务的适配能力，也便于后续扩展新的模型供应商。

6、邀请码准入设计：系统通过邀请码控制用户注册入口。管理员可以生成邀请码，并设置邀请码状态、有效期和使用次数。用户注册时必须通过邀请码校验，从而避免系统被无关用户随意注册，也便于进行内测或企业内部用户管理。

7、套餐与支付设计：系统通过套餐购买和微信支付实现用户权益管理。不同套餐可以对应不同问答额度、Token 额度、知识库容量或模型使用权限。支付成功后，系统自动更新用户套餐状态和使用额度。

8、异步处理设计：文档解析、OCR 识别和向量化属于耗时任务，系统通过 Kafka 进行异步处理，避免用户上传后长时间等待。系统通过状态字段记录处理进度，便于用户查看文件是否已完成入库。

4.2 数据库设计

4.2.1 数据库设计概述

本系统数据库主要用于存储用户信息、组织权限、文件元数据、文档分片、OCR 识别结果、知识片段、模型配置、邀请码、套餐订单和聊天记录等业务数据。系统采用 MySQL 作为主要关系型数据库，用于保存结构化业务数据；Elasticsearch 用于保存文本索引和向量索引；Redis 用于保存短期会话、上传状态和生成状态；MinIO 用于保存用户上传的原始文档、图片和扫描件。

从业务角度看，数据库设计主要围绕以下几类核心数据展开：

数据类别	主要内容	存储位置
用户与权限数据	用户、角色、组织标签、邀请码	MySQL
文件与知识数据	文件信息、分片信息、文本块、OCR 结果	MySQL / Elasticsearch / MinIO
模型配置数据	大语言模型、Embedding 模型、用户模型选择	MySQL
对话数据	会话、消息、引用来源	MySQL / Redis
套餐支付数据	套餐、订单、支付记录、额度	MySQL

临时状态数据	上传进度、短期上下文、生成状态	Redis

表 4-11 数据库设计表

4.2.2 数据库概念结构设计

系统主要实体包括：用户、组织标签、文件、文件分片、知识片段、模型配置、邀请码、套餐、订单、会话和消息。

各实体之间的关系如下：

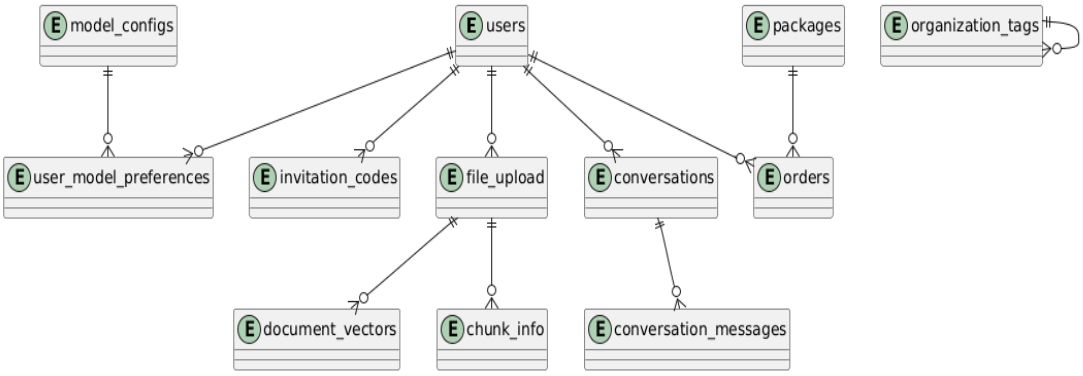

图 4-12 数据库设计实体图

4.2.3 数据库表结构设计

1、用户表 users

users 表用于存储系统用户的基础信息和权限信息。系统通过 role 字段区分普通用户和管理员，通过 org_tags 和 primary_org 字段实现组织标签权限控制。

字段名	数据类型	是否主键	描述
id	BIGINT	是	用户唯一标识
username	VARCHAR(255)	否	用户名，唯一
password	VARCHAR(255)	否	加密后的密码
role	VARCHAR(20)	否	用户角色，USER 或 ADMIN
org_tags	VARCHAR(255)	否	用户所属组织标签，多个用逗号分隔
primary_org	VARCHAR(50)	否	用户主组织标签
created_at	TIMESTAMP	否	创建时间
updated_at	TIMESTAMP	否	更新时间

表 4-13 用户表

2、组织标签表 organization_tags

organization_tags 表用于维护企业内部组织结构，支持父子层级关系，可用

于部门、项目组或业务线的数据隔离。

字段名	数据类型	是否主键	描述
tag_id	VARCHAR(50)	是	组织标签唯一标识
name	VARCHAR(100)	否	组织标签名称
description	TEXT	否	组织标签描述
parent_tag	VARCHAR(50)	否	父级组织标签
created_by	BIGINT	否	创建者用户 ID
created_at	TIMESTAMP	否	创建时间
updated_at	TIMESTAMP	否	更新时间

表 4-14 组织标签表

3、文件上传表 file_upload

file_upload 表用于记录用户上传的文档、图片、截图和扫描件等文件信息。系统通过 file_md5 实现文件去重，通过 org_tag 和 is_public 控制文件访问权限。

字段名	数据类型	是否主键	描述
id	BIGINT	是	文件记录主键
file_md5	VARCHAR(32)	否	文件 MD5 指纹
file_name	VARCHAR(255)	否	原始文件名
file_type	VARCHAR(50)	否	文件类型，如 PDF、DOCX、IMAGE
source_type	VARCHAR(30)	否	来源类型，DOCUMENT、IMAGE、SCAN
total_size	BIGINT	否	文件大小
status	TINYINT	否	文件处理状态
user_id	BIGINT	否	上传用户 ID
org_tag	VARCHAR(50)	否	文件所属组织标签
is_public	BOOLEAN	否	是否公开
storage_path	VARCHAR(255)	否	MinIO 存储路径
created_at	TIMESTAMP	否	创建时间
merged_at	TIMESTAMP	否	文件合并完成时间

表 4-15 文件上传表

4、文件分片表 chunk_info

chunk_info 表用于支持大文件分片上传和断点续传。每个文件可以拆分为多个分片，每个分片独立保存校验信息和存储路径。本项目使用的方法是最大-最小语义分块法该方法利用语义相似性和最大-最小算法来识别语义连贯的文本。我们在三个不同的数据集上评估了该方法，通过调整互信息（AMI）评估聚类效率，并通过基于 RAG 的多项选择题测试的准确率评估生成连贯性[9]。

字段名	数据类型	是否主键	描述
id	BIGINT	是	分片记录主键
file_md5	VARCHAR(32)	否	所属文件 MD5
chunk_index	INT	否	分片序号
chunk_md5	VARCHAR(32)	否	分片 MD5
storage_path	VARCHAR(255)	否	分片在 MinIO 中的路径

表 4-16 文本分片表

5、知识片段表 document_vectors

document_vectors 表用于保存文档解析或 OCR 识别后的文本片段信息。实际向量索引主要存储在 Elasticsearch 中，MySQL 中保存文本内容、来源信息和模型版本，便于追溯和管理。

字段名	数据类型	是否主键	描述
vector_id	BIGINT	是	知识片段主键
file_md5	VARCHAR(32)	否	所属文件 MD5
chunk_id	INT	否	文本片段序号
text_content	LONGTEXT	否	文本片段内容
source_type	VARCHAR(30)	否	来源类型，文档解析或 OCR
model_version	VARCHAR(100)	否	向量模型版本
user_id	BIGINT	否	所属用户 ID
org_tag	VARCHAR(50)	否	所属组织标签
is_public	BOOLEAN	否	是否公开
created_at	TIMESTAMP	否	创建时间

表 4-17 知识片段表

6、模型配置表 model_configs

model_configs 表是本系统的重要设计，用于管理大语言模型和 Embedding 模型配置。管理员可以在后台新增、修改、启用或停用不同模型，使系统能够灵活切换问答模型和向量化模型。

字段名	数据类型	是否主键	描述
id	BIGINT	是	模型配置主键
model_name	VARCHAR(100)	否	模型名称
provider	VARCHAR(100)	否	模型供应商，如 DeepSeek、通义千问等
model_type	VARCHAR(30)	否	模型类型，LLM 或 EMBEDDING
api_url	VARCHAR(255)	否	模型接口地址
api_key	VARCHAR(255)	否	模型调用密钥
model_code	VARCHAR(100)	否	实际调用的模型编码
is_enabled	BOOLEAN	否	是否启用
is_default	BOOLEAN	否	是否为默认模型
created_at	TIMESTAMP	否	创建时间
updated_at	TIMESTAMP	否	更新时间

表 4-18 模型配置表

7、用户模型偏好表 user_model_preferences

user_model_preferences 表用于保存用户选择的默认问答模型或向量模型。系统在用户发起问答时，可根据用户偏好、套餐权限和管理员配置选择实际调用的模型。

字段名	数据类型	是否主键	描述
id	BIGINT	是	主键
user_id	BIGINT	否	用户 ID
llm_model_id	BIGINT	否	用户选择的大语言模型 ID
embedding_model_id	BIGINT	否	用户选择的向量模型 ID
created_at	TIMESTAMP	否	创建时间
updated_at	TIMESTAMP	否	更新时间

表 4-19 用户模型偏好表

8、邀请码表 invitation_codes

invitation_codes 表用于控制用户注册入口。管理员可以生成邀请码，并设置邀请码的有效期、使用次数和启用状态。

字段名	数据类型	是否主键	描述
id	BIGINT	是	邀请码主键
code	VARCHAR(100)	否	邀请码内容，唯一
max_usage	INT	否	最大使用次数
used_count	INT	否	已使用次数
status	VARCHAR(20)	否	邀请码状态
expired_at	TIMESTAMP	否	过期时间
created_by	BIGINT	否	创建者 ID
created_at	TIMESTAMP	否	创建时间
updated_at	TIMESTAMP	否	更新时间

表 4-20 邀请码表

9、邀请码使用记录表 invitation_code_usage

该表用于记录邀请码被哪些用户使用，便于管理员追踪注册来源。

字段名	数据类型	是否主键	描述
id	BIGINT	是	主键
invite_code_id	BIGINT	否	邀请码 ID
user_id	BIGINT	否	使用邀请码注册的用户 ID
used_at	TIMESTAMP	否	使用时间

表 4-21 邀请码使用记录表

10、套餐表 packages

packages 表用于定义系统中的套餐类型，不同套餐可以对应不同的问答额度、Token 额度、知识库容量和可用模型范围。

字段名	数据类型	是否主键	描述
id	BIGINT	是	套餐主键
package_name	VARCHAR(100)	否	套餐名称
price	DECIMAL(10,2)	否	套餐价格
question_quota	INT	否	问答次数额度
token_quota	BIGINT	否	Token 额度
storage_quota	BIGINT	否	存储容量额度
available_models	VARCHAR(255)	否	可用模型 ID 列表
validity_days	INT	否	套餐有效天数
is_enabled	BOOLEAN	否	是否启用
created_at	TIMESTAMP	否	创建时间
updated_at	TIMESTAMP	否	更新时间

表 4-22 套餐表

11、订单表 orders

orders 表用于记录用户购买套餐时产生的订单信息。系统通过订单状态判断用户是否完成支付，并据此开通套餐权益。

字段名	数据类型	是否主键	描述
id	BIGINT	是	订单主键
order_no	VARCHAR(100)	否	订单编号，唯一
user_id	BIGINT	否	下单用户 ID
package_id	BIGINT	否	套餐 ID
amount	DECIMAL(10,2)	否	订单金额
pay_type	VARCHAR(30)	否	支付方式，如 WECHAT
status	VARCHAR(30)	否	订单状态
pay_qr_code	VARCHAR(255)	否	微信支付二维码或收款码地址
paid_at	TIMESTAMP	否	支付完成时间

created_at	TIMESTAMP	否	创建时间
updated_at	TIMESTAMP	否	更新时间

表 4-23 订单表

12、用户套餐权益表 user_subscriptions

user_subscriptions 表用于记录用户当前生效的套餐权益和剩余额度。

字段名	数据类型	是否主键	描述
id	BIGINT	是	主键
user_id	BIGINT	否	用户 ID
package_id	BIGINT	否	套餐 ID
remaining_questions	INT	否	剩余问答次数
remaining_tokens	BIGINT	否	剩余 Token 数
remaining_storage	BIGINT	否	剩余存储容量
started_at	TIMESTAMP	否	生效时间
expired_at	TIMESTAMP	否	过期时间
status	VARCHAR(30)	否	套餐状态

表 4-24 用户套餐权益表

13、会话表 conversations

conversations 表用于保存用户的聊天会话信息。一个用户可以拥有多个会话。

字段名	数据类型	是否主键	描述
id	BIGINT	是	会话主键
user_id	BIGINT	否	用户 ID
title	VARCHAR(255)	否	会话标题
model_id	BIGINT	否	本次会话使用的模型 ID
created_at	TIMESTAMP	否	创建时间
updated_at	TIMESTAMP	否	更新时间

表 4-25 会话表

14、聊天消息表 conversation_messages

conversation_messages 表用于保存用户问题、模型回答和引用来源信息。

字段名	数据类型	是否主键	描述
id	BIGINT	是	消息主键
conversation_id	BIGINT	否	所属会话 ID
role	VARCHAR(30)	否	消息角色，user 或 assistant
content	LONGTEXT	否	消息内容
reference_mappings	JSON	否	引用来源信息
token_usage	INT	否	本条消息消耗 Token
created_at	TIMESTAMP	否	创建时间

表 4-26 聊天消息表

4.3 系统功能设计

本系统围绕多模态 RAG 智能知识库的核心业务流程进行功能设计，主要包括用户管理、文件上传与解析、OCR 识别、知识库检索、聊天助手、模型配置与选择、邀请码管理、套餐支付和权限管理等模块。各模块之间相互配合，共同完成从用户准入、资料入库、知识检索到智能问答的完整流程。

4.3.1 用户管理模块设计

用户管理模块负责用户注册、登录、身份认证和基础权限控制，是系统安全访问的基础。系统通过用户名和密码完成身份认证，通过 JWT Token 保持登录状态，通过角色和组织标签控制功能权限与数据访问范围。

设计项	内容
功能目的	实现用户注册、登录、身份认证和用户信息管理
输入数据	用户名、密码、邀请码、用户角色、组织标签
处理过程	校验邀请码，检查用户名是否存在，对密码进行加密，创建用户记录，生成 JWT Token

输出结果	注册结果、登录 Token、用户基本信息、用户权限信息

表 4-27 用户管理模块设计表

用户注册时，系统首先校验邀请码是否有效。如果邀请码不存在、已过期、已停用或超过使用次数，则不允许注册。校验通过后，系统检查用户名是否重复，并使用 BCrypt 对密码进行加密存储。注册成功后，系统为用户创建基础信息，并绑定默认角色和初始权限。

用户登录时，系统根据用户名查询用户信息，验证密码是否正确。验证通过后，系统生成包含用户 ID、角色、组织标签等信息的 JWT Token，并返回给前端。后续请求通过请求头携带 Token，后端解析 Token 后完成用户身份识别。

4.3.2 邀请码管理模块设计

邀请码管理模块用于控制系统注册入口，适合系统内测、企业内部使用或限定范围开放场景。管理员可以创建邀请码，并设置邀请码的有效期、最大使用次数和启用状态。

设计项	内容
功能目的	控制用户注册入口，提高用户准入管理能力
输入数据	邀请码、有效期、最大使用次数、启用状态
处理过程	管理员创建邀请码，用户注册时系统校验邀请码状态、有效期和使用次数
输出结果	邀请码创建结果、校验结果、邀请码使用记录

表 4-28 邀请码管理模块设计表

邀请码系统的主要流程为：管理员在后台生成邀请码；用户注册时填写邀请码；系统校验邀请码是否有效；校验通过后创建用户账号，并记录邀请码使用情况。该模块可以避免系统被无关用户随意注册，也便于后续追踪用户来源。

4.3.3 文件上传与解析模块设计

文件上传与解析模块是知识入库流程的起点。系统支持 Word、PDF、TXT、Excel 等常见文档格式，也支持图片、截图和扫描件等多模态资料。对于较大的文件，系统采用分片上传和断点续传机制，以提高上传稳定性。

设计项	内容
功能目的	实现文档、图片、扫描件等资料上传，并完成基础解析
输入数据	文件 MD5、文件名、文件大小、分片序号、组织标签、公开状态、文件内容
处理过程	客户端计算文件 MD5，服务端接收分片并存入 MinIO，Redis 记录上传状态，分片完成后合并文件，并发送异步解析任务
输出结果	文件上传状态、文件存储路径、解析任务状态

表 4-29 文件上传与解析模块设计表

系统在上传前由前端计算文件 MD5，用于文件唯一性判断和断点续传。服务端接收分片后，将分片文件保存到 MinIO，并使用 Redis Bitmap 记录已上传分片。当所有分片上传完成后，系统将分片合并为完整文件，并在 MySQL 中记录文件元数据。

文件合并完成后，系统通过 Kafka 发送异步处理任务。对于普通文档，系统调用文档解析工具提取文本内容；对于图片、截图和扫描件，则进入 OCR 识别流程。该设计可以避免大文件上传和解析过程长时间阻塞用户操作。

4.3.4 OCR 识别模块设计

OCR 识别模块用于处理图片、截图和扫描件中的文字内容，是本系统支持多模态知识入库的重要功能。通过 OCR，系统可以将原本无法直接检索的图片资料转化为可切片、可向量化、可检索的文本知识。

设计项	内容
功能目的	提取图片、截图、扫描件中的文字内容
输入数据	图片文件、扫描件文件、文件 ID、文件来源信息
处理过程	系统读取图片文件，调用 OCR 服务识别文字，对识别结果进行清洗和格式化

输出结果	OCR 识别文本、识别状态、识别来源信息

表 4-30 OCR 识别模块设计表

OCR 识别完成后，系统会将识别出的文本与原始文件建立关联，并保留文件来源信息。识别结果后续会进入文本切片和向量化流程，与普通文档解析出的文本采用统一的知识入库方式。

对于 OCR 识别后的内容，系统仍然继承原始文件的组织标签和访问权限，确保图片资料转化为文本后不会绕过权限控制。

4.3.5 向量化与索引模块设计

向量化与索引模块负责将文档解析结果和 OCR 识别结果转化为可检索的知识片段。系统先对文本进行清洗和切片，再调用 Embedding 模型生成向量，最后写入 Elasticsearch 索引。

设计项	内容
功能目的	将文本内容转化为可用于语义检索的向量数据
输入数据	文本内容、文件 MD5、文本块序号、模型配置、权限信息
处理过程	文本清洗、文本切片、选择 Embedding 模型、调用向量化接口、写入 Elasticsearch
输出结果	文本块、向量数据、索引记录、处理状态

表 4-31 向量化与索引模块设计表

系统会根据一定规则对长文本进行切分，避免单个文本块过长影响检索效果。每个文本块会保存对应的文件来源、块序号、组织标签、是否公开、向量模型版本等信息。

向量化时，系统会根据管理员配置的模型信息选择对应的 Embedding 模型。向量数据主要写入 Elasticsearch，用于后续语义检索；MySQL 中保存文本块元数据和处理状态，方便管理和追溯。

4.3.6 知识库检索模块设计

知识库检索模块是系统实现 RAG 问答的基础。系统采用关键词检索与语义

检索结合的混合检索方式，既支持精确关键词匹配，也支持自然语言语义匹配。

设计项	内容
功能目的	根据用户问题从知识库中检索相关知识片段
输入数据	用户查询内容、topK 参数、用户 ID、组织标签、权限信息
处理过程	查询文本向量化，构造 Elasticsearch 查询条件，执行关键词检索和语义检索，进行权限过滤和结果排序
输出结果	相关知识片段、相似度得分、来源文件信息

表 4-32 知识库检索模块设计表

检索过程首先将用户查询文本转化为查询向量，然后结合关键词匹配和向量相似度计算执行混合检索。系统会在检索条件中加入权限过滤规则，确保用户只能检索到本人上传、公开共享或所属组织范围内的知识内容。

检索结果会根据相关性得分进行排序，并返回排名靠前的知识片段。每条结果包含文本内容、文件来源、片段编号和得分信息，为后续构造 RAG Prompt 提供依据。

4.3.7 聊天助手模块设计

聊天助手模块是用户与系统交互的核心模块。用户通过自然语言输入问题，系统结合知识库检索结果和历史上下文，调用大语言模型生成回答，并通过 WebSocket 实时返回。

设计项	内容
功能目的	实现基于知识库的智能问答和多轮对话
输入数据	用户问题、会话 ID、用户选择模型、历史对话、检索结果
处理过程	建立 WebSocket 连接，接收用户问题，检索知识片段，构造 Prompt，调用大语言模型，流式返回回答，保存对话记录

输出结果	模型回答、引用来源、聊天记录、会话状态

表 4-33 聊天助手模块设计表

聊天助手模块的核心流程如下：

用户进入聊天页面并建立 WebSocket 连接。

用户输入问题并发送到后端。

系统根据问题执行知识库混合检索。

系统读取 Redis 中的历史上下文。

系统将用户问题、历史记录和检索结果组合成 Prompt。

系统调用用户选择的大语言模型。

模型生成内容后，通过 WebSocket 流式返回给前端。

系统保存用户问题、模型回答和引用来源。

该模块支持多轮对话，Redis 用于保存短期上下文，MySQL 用于保存长期聊天记录。系统还支持停止生成和断线重连，提升用户交互体验。

4.3.8 Prompt 构建与管理模块设计

Prompt 构建与管理模块用于组织发送给大语言模型的上下文内容。系统会根据用户问题、检索结果、历史对话和系统规则动态生成 Prompt。

设计项	内容
功能目的	构造适合大语言模型理解的输入内容
输入数据	用户问题、知识片段、历史对话、Prompt 模板
处理过程	选择模板，填充检索上下文，拼接历史对话，控制长度，生成最终 Prompt
输出结果	可发送给大语言模型的 Prompt 内容

表 4-34 Prompt 构建与管理模块设计表

Prompt 通常包含系统角色说明、回答规则、检索到的参考资料、对话历史和用户问题。通过 Prompt 管理，系统可以引导模型优先基于知识库内容回答，并在回答中体现来源依据。

4.3.9 模型配置模块设计

模型配置模块用于管理系统接入的大语言模型和 Embedding 模型。管理员可以在后台维护模型名称、供应商、模型类型、接口地址、API Key、启用状态

和默认状态等信息。

设计项	内容
功能目的	支持系统灵活接入和切换不同模型服务
输入数据	模型名称、模型类型、接口地址、API Key、模型编码、启用状态
处理过程	管理员新增或编辑模型配置，系统校验配置有效性，保存模型信息，供问答和向量化模块调用
输出结果	模型配置记录、默认模型、可用模型列表

表 4-35 模型配置模块设计表

模型配置模块使系统不固定依赖单一模型服务。大语言模型用于智能问答，Embedding 模型用于文本向量化。管理员可以根据实际情况启用、停用或切换模型，提高系统的可维护性和扩展性。

4.3.10 模型选择模块设计

模型选择模块用于支持用户在问答时选择可用的大语言模型。系统会根据用户套餐、权限和管理员配置，展示用户可使用的模型列表。

设计项	内容
功能目的	支持用户根据需求选择不同模型进行问答
输入数据	用户 ID、套餐信息、模型配置、用户选择结果
处理过程	系统查询用户可用模型列表，用户选择模型，系统在问答时调用对应模型
输出结果	用户当前使用模型、模型回答结果、模型调用记录

表 4-36 模型选择模块设计表

当用户发起问答请求时，系统会检查用户是否具备所选模型的使用权限。如果具备权限，则调用对应模型；如果没有权限，则提示用户切换模型或升级套餐。该功能使系统能够支持不同模型能力分层和套餐权益控制。

4.3.11 套餐与微信支付模块设计

套餐与微信支付模块用于支持用户购买系统服务权益。不同套餐可以配置不同的问答次数、Token 额度、知识库存储容量和可用模型范围。

设计项	内容
功能目的	实现套餐购买、微信扫码支付和用户权益管理
输入数据	用户 ID、套餐 ID、订单金额、支付方式
处理过程	用户选择套餐，系统创建订单，生成微信支付二维码，用户扫码支付，系统确认支付状态并开通权益
输出结果	订单记录、支付状态、用户套餐权益、剩余额度

表 4-37 套餐与微信支付模块设计表

套餐购买成功后，系统更新用户套餐状态，并记录剩余问答次数、Token 额度和模型权限。用户在使用问答或模型能力时，系统会根据套餐规则进行额度扣减。

4.3.12 权限管理模块设计

权限管理模块用于控制用户对系统功能和知识资源的访问范围。系统同时支持角色权限和组织标签权限。

设计项	内容
功能目的	实现系统功能权限和知识数据权限控制
输入数据	用户角色、组织标签、资源组织标签、资源可见性
处理过程	解析 JWT，获取用户角色和组织标签，判断用户是否有权限访问功能或资源
输出结果	允许访问或拒绝访问

表 4-38 权限管理模块设计表

系统权限控制主要包括三类：

角色权限：区分普通用户和管理员。

组织权限：用户只能访问所属组织范围内的资料。

资源可见性：支持私有、组织共享和公开资源。

OCR 识别结果、文本切片和向量索引均继承原始文件权限，保证从上传到检索再到问答的全链路权限一致。

4.3.13 聊天记录模块设计

聊天记录模块用于保存用户与系统之间的问答内容，方便用户后续查看历史会话。

设计项	内容
功能目的	保存和查询用户历史对话
输入数据	会话 ID、用户问题、模型回答、引用来源、时间
处理过程	系统在问答完成后保存消息记录，并支持按用户、时间范围查询
输出结果	历史会话列表、历史消息内容、引用来源信息

表 4-39 聊天记录模块设计表

系统使用 Redis 保存短期对话上下文，用于多轮对话；使用 MySQL 保存完整聊天记录，用于长期查询和追溯。管理员可根据权限查看系统中的会话记录，普通用户只能查看自己的历史记录。

4.3.14 系统管理模块设计

系统管理模块主要面向管理员，用于维护系统中的用户、文件、组织标签、模型配置、邀请码、套餐订单和日志信息。

设计项	内容
功能目的	提供系统后台管理能力
输入数据	用户信息、组织标签、模型配置、邀请码、套餐、订单
处理过程	管理员进行查询、新增、编辑、删除、启用、停用等操作
输出结果	管理操作结果、系统配置结果、统计信息

表 4-40 系统管理模块设计表

系统管理模块主要包括用户管理、组织标签管理、文件管理、模型配置管理、邀请码管理、套餐管理、订单管理和系统日志查看等功能。该模块为系统运行维护和业务管理提供支撑。

4.3.15 核心功能交互流程

1、资料上传与知识入库流程：

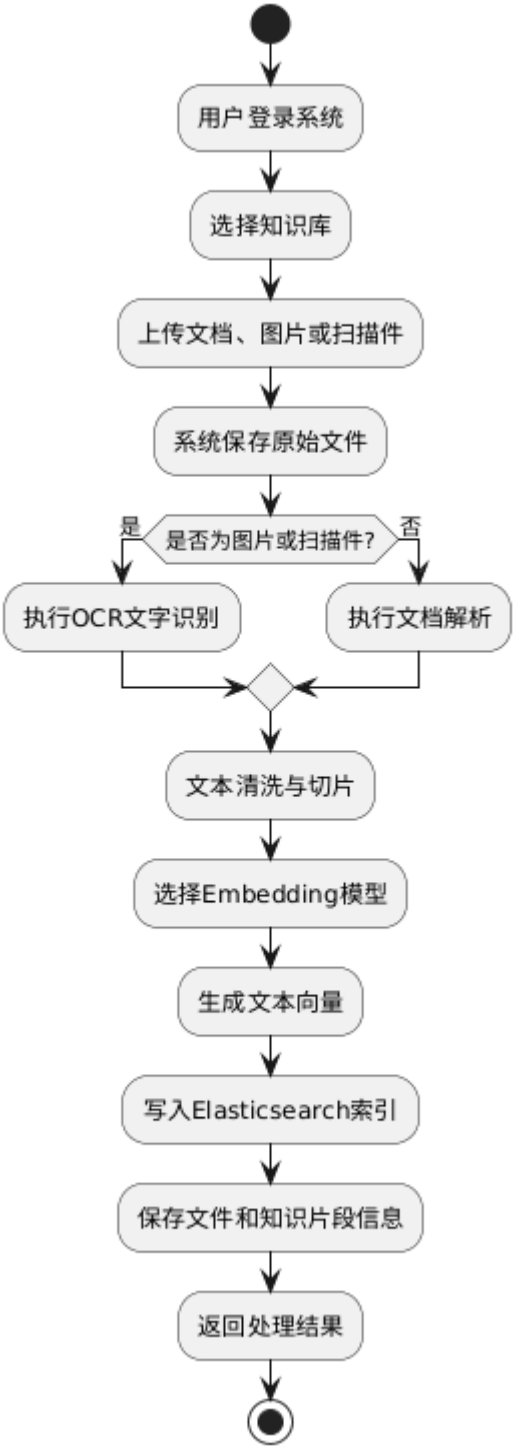

图 4-41　资料上传与知识入库流程图

2、智能问答流程：

图 4-42　智能问答流程图

3、邀请码注册流程

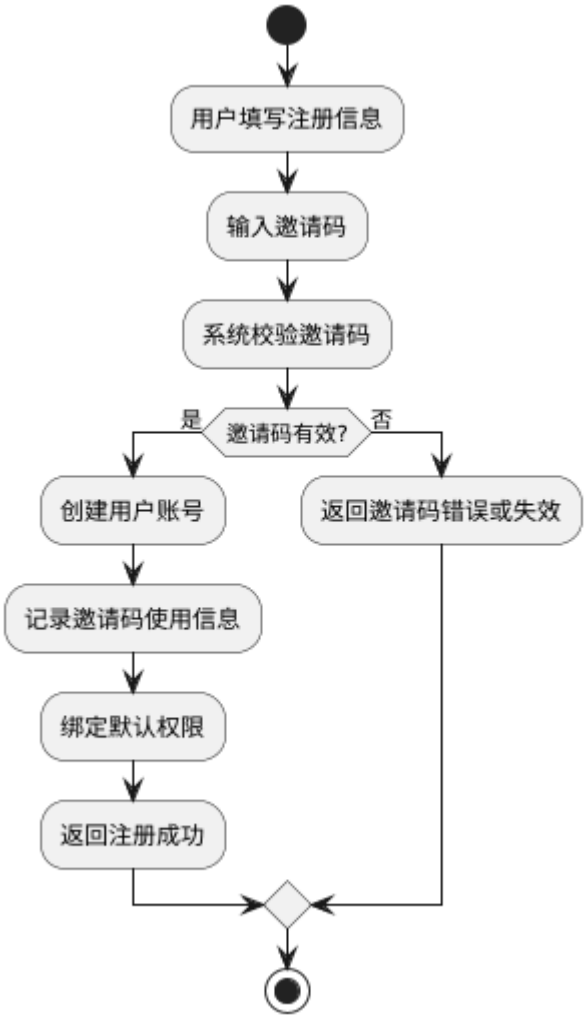

图 4-43 邀请码注册流程图

4、模型配置与选择流程

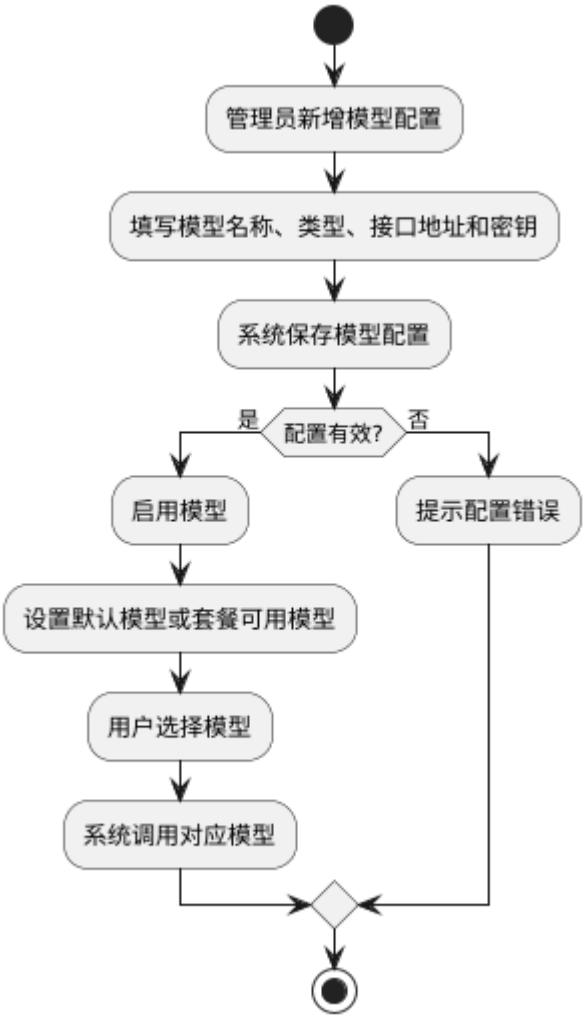

图 4-44 模型配置与选择流程图

5、套餐购买与支付流程

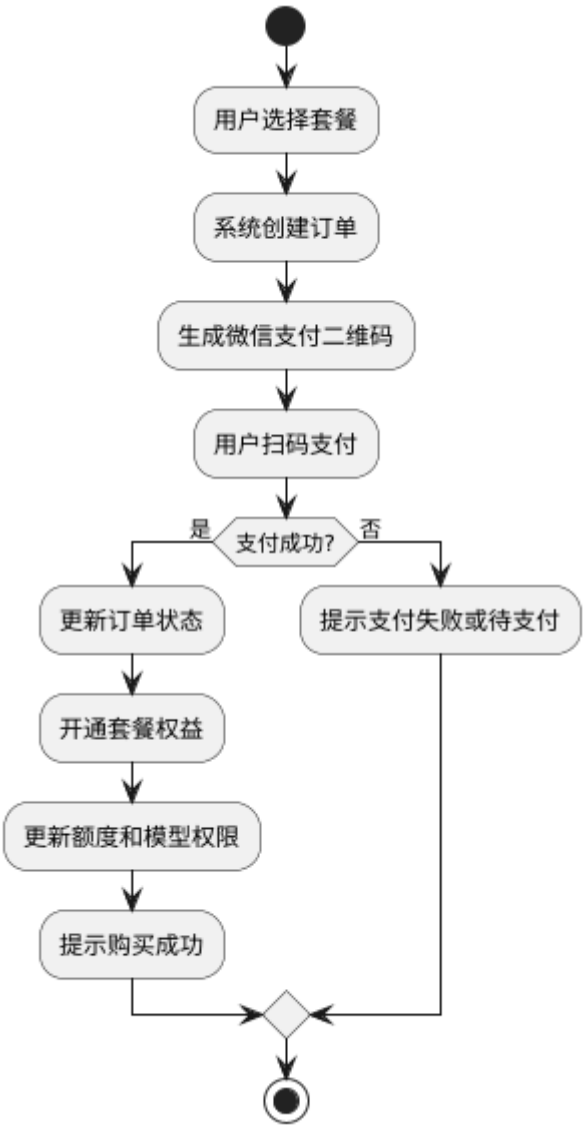

图 4-45　套餐购买与支付流程图

第五章 系统实现

5.1 系统实现概述

本系统是一个基于多模态 RAG 的企业智能知识库系统，采用前后端分离架构实现。后端基于 Spring Boot 3.x 和 Java 17 开发，主要负责用户认证、邀请码管理、文件上传、OCR 识别、文档解析、向量化、知识检索、智能问答、模型配置、套餐购买和微信支付等业务逻辑。前端基于 Vue 3、TypeScript、Vite 和 Naive UI 实现，主要提供知识库管理、智能问答、模型配置、邀请码管理、套餐充值和后台管理等页面。

系统在数据存储方面采用 MySQL、Redis、Elasticsearch 和 MinIO 协同工作。MySQL 用于保存用户、文件、模型配置、邀请码、订单、聊天记录等结构化数据；Redis 用于保存登录缓存、上传分片状态、短期对话上下文和生成状态；Elasticsearch 用于保存文本索引和向量索引，支撑关键词检索与语义检索；MinIO 用于保存用户上传的文档、图片、截图和扫描件等原始文件。系统还通过 Kafka 对文档解析、OCR 识别和向量化等耗时任务进行异步处理，避免长时间阻塞用户请求。

在智能问答方面，系统采用 RAG 技术路线。用户输入问题后，系统先从知识库中进行混合检索，再将检索到的知识片段、历史对话和用户问题共同构造成 Prompt，最后调用大语言模型生成回答。回答内容通过 WebSocket 流式返回给前端，并保存对话记录和引用来源。

5.1.1 实现环境

类型	环境
后端语言	Java 17
后端框架	Spring Boot 3.x
前端框架	Vue 3 + TypeScript
构建工具	Maven、Vite、pnpm
数据库	MySQL 8.0
缓存	Redis
搜索引擎	Elasticsearch
对象存储	MinIO
消息队列	Kafka

文档解析	Apache Tika
实时通信	WebSocket
AI 能力	大语言模型、Embedding、OCR、多模态模型
支付能力	微信扫码支付

表 5-1 实现环境表

5.1.2 后端目录结构

```
src/main/java/com/yizhaoqi/smartpai
├── client          # 外部模型和接口客户端，如 Embedding、Vision Embedding 等
├── config          # Spring Security、JWT、WebSocket、Kafka、MinIO 等配置
├── consumer        # Kafka 消费者，处理文件解析和向量化任务
├── controller      # REST 接口控制层
├── entity          # 部分数据实体
├── exception       # 自定义异常和异常处理
├── handler         # WebSocket 消息处理器
├── model           # JPA 实体和领域模型
├── repository      # 数据访问层
├── service         # 核心业务服务层
├── test            # 测试相关代码
└── utils           # 工具类，如 JWT、加密、文件处理等
```

5.1.3 前端目录结构

```
frontend/src
├── assets          # 静态资源
├── components      # 公共组件
├── constants       # 常量定义
├── enum            # 枚举定义
├── hooks           # 组合式函数
├── layouts         # 页面布局
├── locales         # 国际化配置
├── plugins         # 插件配置
```

```
├── router          # 路由配置
├── service         # API 请求封装
├── store           # Pinia 状态管理
├── styles          # 全局样式
├── theme           # 主题配置
├── typings         # TypeScript 类型声明
├── utils           # 工具函数
├── vendor          # 第三方封装
└── views           # 页面视图
    ├── chat                # 智能问答页面
    ├── chat-history        # 聊天历史页面
    ├── invite-code         # 邀请码管理页面
    ├── knowledge-base      # 知识库管理页面
    ├── model-provider      # 模型配置页面
    ├── org-tag             # 组织标签页面
    ├── personal-center     # 个人中心
    ├── recharge            # 套餐购买页面
    ├── recharge-manage     # 套餐管理页面
    ├── usage-monitor       # 用量监控页面
    └── user                # 用户管理页面
```

图 5-2 实现环境图

图 5-3　登录后首页图

5.2 功能模块实现

5.2.1 用户注册与登录模块实现

用户管理模块主要实现用户注册、登录和身份认证。用户注册时，系统会校验邀请码、检查用户名是否重复，并对密码进行加密保存。注册成功后，系统为用户分配默认角色和私人组织标签。用户登录成功后，系统生成 JWT Token，前端后续请求通过 Token 完成身份认证。

```
@Transactional
public void registerUser(String username, String password, String inviteCode) {
validateRegistrationPolicy(username, inviteCode);
validatePassword(password);

if (userRepository.findByUsername(username).isPresent()) {
    throw new CustomException("Username already exists", HttpStatus.BAD_REQUEST);
}
ensureDefaultOrgTagExists();
User user = new User();
user.setUsername(username);
user.setPassword(PasswordUtil.encode(password));
user.setRole(User.Role.USER);
userRepository.save(user);
String privateTagId = PRIVATE_TAG_PREFIX + username;
```

```
    createPrivateOrgTag(privateTagId, username, user);

    List<String> assignedOrgTags = List.of(DEFAULT_ORG_TAG, privateTagId)
;
    user.setOrgTags(String.join(",", assignedOrgTags));
    user.setPrimaryOrg(privateTagId);
    userRepository.save(user);

    orgTagCacheService.cacheUserOrgTags(username, assignedOrgTags);
    orgTagCacheService.cacheUserPrimaryOrg(username, privateTagId);
    }

    private void validateRegistrationPolicy(String username, String inviteCode)
{
    RegistrationMode mode = appAuthProperties.getRegistration().getMode();
    boolean inviteRequired = appAuthProperties.getRegistration().isInviteRequired
()
    || mode == RegistrationMode.INVITE_ONLY;

    if (mode == RegistrationMode.CLOSED) {
        throw new CustomException("REGISTRATION_CLOSED", HttpStatus.
FORBIDDEN);
    }

    if (inviteRequired) {
        inviteCodeService.consume(inviteCode, username);
    }
    }
```

图 5-4　注册页面图

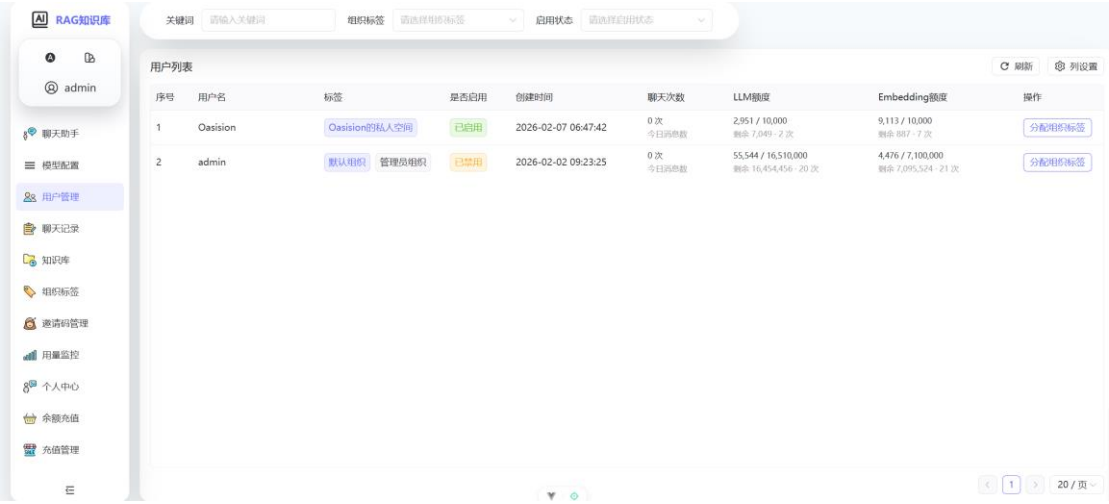

图 5-5　用户管理图

5.2.2 邀请码管理模块实现

邀请码管理模块用于控制用户注册入口。管理员可以生成邀请码、设置最大使用次数、设置过期时间、停用邀请码和删除邀请码。用户注册时，系统通过邀请码服务校验邀请码是否可用，并更新已使用次数。

```
@Transactional
public List<InviteCode> createInviteCodes(String creatorUsername,
                                          String requestedCode,
                                          Integer maxUses,
                                          LocalDateTime expiresAt,
                                          Integer count) {
```

```
        User creator = userRepository.findByUsername(creatorUsername)
                .orElseThrow(() -> new CustomException("Creator not found",
 HttpStatus.NOT_FOUND));

        if (creator.getRole() != User.Role.ADMIN) {
            throw new CustomException("Only administrators can create invit
e codes", HttpStatus.FORBIDDEN);
        }
        int normalizedCount = count == null || count <= 0 ? 1 : count;
        int normalizedMaxUses = maxUses == null || maxUses <= 0 ? 1 : m
axUses;
        List<InviteCode> inviteCodes = new ArrayList<>();
        Set<String> generatedCodes = new HashSet<>();
        for (int i = 0; i < normalizedCount; i++) {
            InviteCode inviteCode = new InviteCode();
            String code = resolveInviteCode(requestedCode, normalizedCount,
generatedCodes);
            inviteCode.setCode(code);
            inviteCode.setMaxUses(normalizedMaxUses);
            inviteCode.setUsedCount(0);
            inviteCode.setExpiresAt(expiresAt);
            inviteCode.setEnabled(true);
            inviteCode.setCreatedBy(creator);
            inviteCodes.add(inviteCode);
            generatedCodes.add(code);
        }
        return inviteCodeRepository.saveAll(inviteCodes);
    }
    @Transactional
    public void consume(String code, String username) {
        if (code == null || code.isBlank()) {
            throw new CustomException("INVITE_CODE_REQUIRED", HttpS
tatus.FORBIDDEN);
        }
        InviteCode inviteCode = inviteCodeRepository.findByCodeForUpdate(no
rmalizeCode(code))
                .orElseThrow(() -> new CustomException("INVITE_CODE_IN
```

```
VALID", HttpStatus.FORBIDDEN));

        if (!Boolean.TRUE.equals(inviteCode.getEnabled())) {
            throw new CustomException("INVITE_CODE_INVALID", HttpStat
us.FORBIDDEN);
        }
        if (inviteCode.getExpiresAt() != null
                && inviteCode.getExpiresAt().isBefore(LocalDateTime.now()))
 {
            throw new CustomException("INVITE_CODE_EXPIRED", HttpSta
tus.FORBIDDEN);
        }
        if (inviteCode.getUsedCount() >= inviteCode.getMaxUses()) {
            throw new CustomException("INVITE_CODE_EXHAUSTED", Htt
pStatus.FORBIDDEN);
        }
        inviteCode.setUsedCount(inviteCode.getUsedCount() + 1);
        inviteCodeRepository.save(inviteCode);
    }
```

图 5-6 邀请码管理界面图

5.2.3 文件上传与解析模块实现

文件上传与解析模块支持大文件分片上传、断点续传、文件合并和异步解析。前端计算文件 MD5 后，将文件拆分为多个分片上传。后端接收分片后，将分片保存到 MinIO，并通过 Redis 记录分片上传状态。全部分片上传完成后，系

统将分片合并为完整文件，并发送 Kafka 任务进行文档解析和向量化处理。

```
    public void uploadChunk(String fileMd5,
                            int chunkIndex,
                            long totalSize,
                            String fileName,
                            MultipartFile file,
                            String orgTag,
                            boolean isPublic,
                            String userId) throws IOException {
        FileUpload fileUpload = getOrCreateFileUpload(
                fileMd5, totalSize, fileName, orgTag, isPublic, userId, detectF
ileType(fileName)
        );

        if (fileUpload.getStatus() == FileUpload.STATUS_MERGING) {
            throw new CustomException("文件正在合并中，请稍后重试", Htt
pStatus.CONFLICT);
        }
        if (fileUpload.getStatus() == FileUpload.STATUS_COMPLETED) {
            throw new CustomException("文件已完成合并，不允许继续上传分
片", HttpStatus.CONFLICT);
        }

        if (isChunkUploaded(fileMd5, chunkIndex, userId)) {
            return;
        }

        byte[] fileBytes = file.getBytes();
        String chunkMd5 = DigestUtils.md5Hex(fileBytes);
        String storagePath = "chunks/" + fileMd5 + "/" + chunkIndex;

        minioClient.putObject(PutObjectArgs.builder()
                .bucket("uploads")
                .object(storagePath)
                .stream(new ByteArrayInputStream(fileBytes), file.getSize(), -1
)
                .contentType(file.getContentType())
```

```
                .build());

        saveChunkInfo(fileMd5, chunkIndex, chunkMd5, storagePath);
        markChunkUploaded(fileMd5, chunkIndex, userId);
    }
```

图 5-7 知识库文件上传界面图

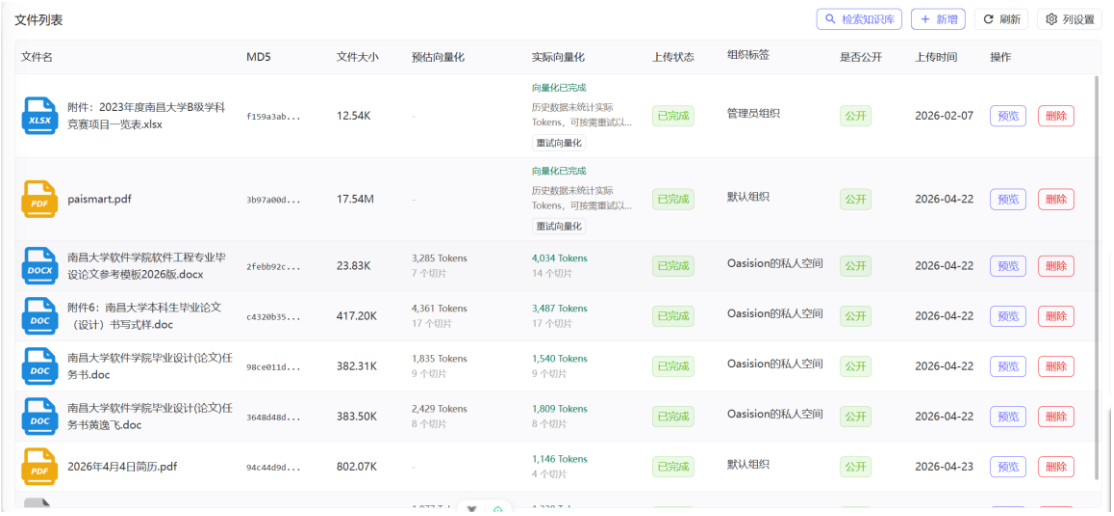

图 5-8 文件上传处理界面图

5.2.4 OCR 与多模态资料处理模块实现

OCR 模块用于识别图片、截图和扫描件中的文字。系统支持 `jpg、jpeg、png、bmp、gif、tif、tiff、webp` 等图片格式。上传图片类文件后，系统调用 OCR 服务提取文字，并将识别结果进入统一的文本切片和向量化流程。对于扫描版 PDF，系统可在普通文本提取不足时进行 OCR fallback，从而提高扫描件资

料的可用性。

```
public boolean isImageFile(String fileName) {
    String extension = extractExtension(fileName);
    return extension != null && OCR_IMAGE_EXTENSIONS.contains(extension);
}
public String extractTextFromImage(InputStream imageStream, String sourceName)
        throws IOException, TikaException {
    OcrProviderSettings providerSettings = resolveProviderSettings();
    ensureOcrReady("Image OCR", providerSettings);
    try {
        RecognizeGeneralRequest request = new RecognizeGeneralRequest();
        request.setBody(imageStream);
        RuntimeOptions runtimeOptions = buildRuntimeOptions();
        RecognizeGeneralResponse response =
                createClient(providerSettings).recognizeGeneralWithOptions(request, runtimeOptions);
        String text = extractTextFromResponse(response);
        return normalizeOcrText(text);
    } catch (Exception e) {
        throw new TikaException("Alibaba OCR failed: " + e.getMessage(), e);
    }
}
```

图 5-9 OCR 图片上传与识别结果界面图

5.2.5 向量化与知识索引模块实现

向量化模块负责将文档解析结果和 OCR 识别结果转化为向量表示。系统根据文件类型选择文本 Embedding 或视觉 Embedding。普通文本内容调用文本 Embedding 模型，图片类内容可调用视觉向量模型。向量化完成后，系统将文本内容、向量、文件来源、组织标签和权限信息写入 Elasticsearch。

```
    public VectorizationUsageResult vectorizeWithUsage(String fileMd5,
                                                       List<String> chu
nks,
                                                       String userId) {
        FileUpload fileUpload = fileUploadRepository
                .findFirstByFileMd5OrderByCreatedAtDesc(fileMd5)
                .orElse(null);

        boolean imageFile = fileUpload != null && ocrService.isImageFile(file
Upload.getFileName());
        int actualEmbeddingTokens = 0;

        for (int i = 0; i < chunks.size(); i++) {
            String chunk = chunks.get(i);

            if (imageFile) {
                VisionEmbeddingClient.EmbeddingUsageResult visionResult =
                        visionEmbeddingClient.embedImage(fileUpload, Visio
nEmbeddingClient.UsageType.UPLOAD);
                actualEmbeddingTokens += visionResult.totalTokens();
            } else {
                EmbeddingClient.EmbeddingUsageResult embeddingResult =
                        embeddingClient.embedWithUsage(List.of(chunk), use
rId, EmbeddingClient.UsageType.UPLOAD);
                actualEmbeddingTokens += embeddingResult.totalTokens();
            }

            // 写入 Elasticsearch 的逻辑包含文本、向量、文件来源和权限信
息
        }

        return new VectorizationUsageResult(actualEmbeddingTokens, chunks.siz
e(), currentModelVersion());
    }
```

paismart.pdf	3b97a00d...	17.54M	-	历史数据未统计实际Tokens，可按需重试以... 重试向量化	已完成	默认组织	公开	2026-04-22	预览 删除
南昌大学软件学院软件工程专业毕设论文参考模板2026版.docx	2febb92c...	23.83K	3,285 Tokens 7 个切片	4,034 Tokens 14 个切片	已完成	Oasision的私人空间	公开	2026-04-22	预览 删除
附件6：南昌大学本科生毕业论文（设计）书写式样.doc	c4320b35...	417.20K	4,361 Tokens 17 个切片	3,487 Tokens 17 个切片	已完成	Oasision的私人空间	公开	2026-04-22	预览 删除
南昌大学软件学院毕业设计(论文)任务书.doc	98ce011d...	382.31K	1,835 Tokens 9 个切片	1,540 Tokens 9 个切片	已完成	Oasision的私人空间	公开	2026-04-22	预览 删除
南昌大学软件学院毕业设计(论文)任务书黄逸飞.doc	3648d48d...	383.50K	2,429 Tokens 8 个切片	1,809 Tokens 8 个切片	已完成	Oasision的私人空间	公开	2026-04-22	预览 删除
2026年4月4日简历.pdf	94c44d9d...	802.07K	-	1,146 Tokens 4 个切片	已完成	默认组织	公开	2026-04-23	预览 删除

图 5-10 OCR 文档向量化完成界面图

5.2.6 知识库混合检索模块实现

知识库检索模块采用关键词检索与语义检索结合的混合检索方式。系统接收到用户问题后，先将问题向量化，再在 Elasticsearch 中执行向量检索和全文检索，并结合组织标签进行权限过滤。对于涉及图片资料的问题，系统会根据路由策略补充视觉召回结果，从而支持多模态 RAG 检索。

```
public List<SearchResult> searchWithPermission(String query, String userId, int topK) {
    try {
        List<String> userEffectiveTags = getUserEffectiveOrgTags(userId);
        String userDbId = getUserDbId(userId);

        List<SearchResult> mergedResults = new ArrayList<>();
        mergedResults.addAll(
                searchTextWithPermission(query, userId, userDbId, userEffectiveTags, topK)
        );

        if (multimodalRoutingService.shouldRunVisionRecall(query)) {
            mergedResults.addAll(
                    searchVisionWithPermission(query, userId, userDbId, userEffectiveTags, topK)
            );
        }

        attachFileNames(mergedResults);
        return mergeAndLimitResults(mergedResults, topK, MAX_CHUNKS_PER_FILE);
    } catch (Exception exception) {
```

```
            logger.error("Permission-aware hybrid search failed", exception);
            return Collections.emptyList();
        }
    }
```

图 5-11　知识库检索结果界面图

5.2.7 智能问答模块实现

智能问答模块通过 WebSocket 实现流式响应。用户输入问题后，系统获取会话上下文，执行知识库混合检索，构造 RAG 上下文，并根据是否存在图片资料决定调用普通大语言模型或多模态大语言模型。模型生成内容后，系统将文本片段实时推送到前端，同时把完整对话保存到历史记录中。

```
public void processMessage(String userId, String userMessage, WebSocketSession session) {
        String conversationId = getOrCreateConversationId(userId);
        GenerationSnapshot generation =
                chatGenerationStateService.createGeneration(userId, conversationId, userMessage);

        List<Map<String, String>> history = getConversationHistory(conversationId);
        List<SearchResult> searchResults = searchService.searchWithPermission(userMessage, userId, 5);

        String context = buildContext(searchResults, generation.generationId());
```

```
        List<ImageAsset> imageAssets = loadImageAssets(searchResults);

        boolean useMultimodal = !imageAssets.isEmpty()
                && multimodalRoutingService.shouldUseMultimodalGeneration
(userMessage, searchResults);

        ChatStreamHandle streamHandle = useMultimodal
                ? multimodalLlmProviderRouter.streamResponse(
                        userId, userMessage, context, history, imageAssets,
                        chunk -> sendResponseChunk(userId, generation.gene
rationId(), conversationId, chunk),
                        error -> handleError(userId, generation.generationId(),
 error))
                : llmProviderRouter.streamResponse(
                        userId, userMessage, context, history,
                        chunk -> sendResponseChunk(userId, generation.gene
rationId(), conversationId, chunk),
                        error -> handleError(userId, generation.generationId(),
 error));

        activeStreams.put(generation.generationId(), streamHandle);
    }
```

图 5-12　智能问答界面图

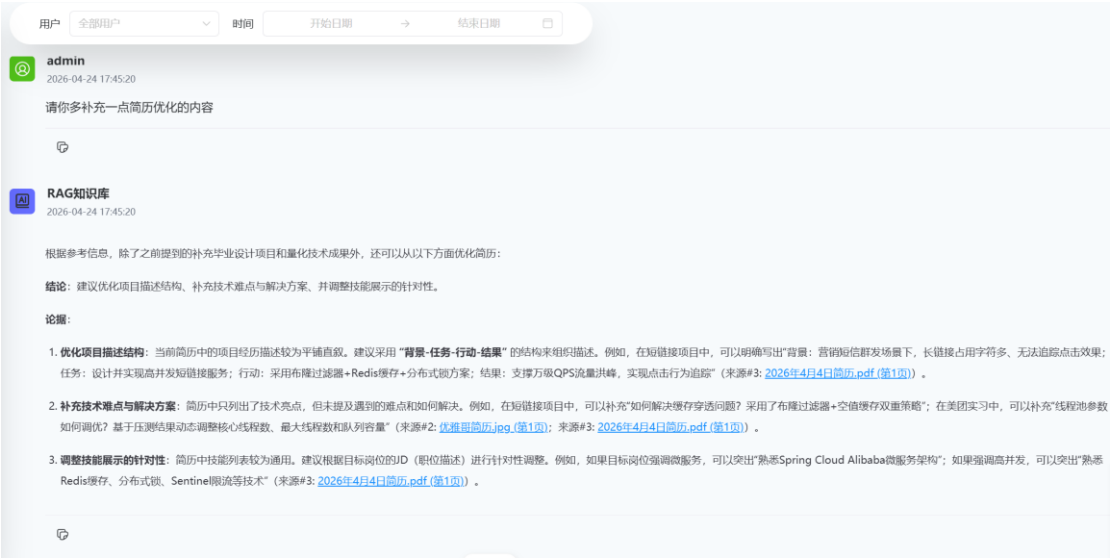

图 5-13　聊天历史界面图

5.2.8 模型配置与模型选择模块实现

模型配置模块用于管理 LLM、Embedding、OCR、Vision Embedding 和 Multimodal LLM 等不同类型的模型 Provider。管理员可以在后台配置模型接口地址、模型名称、密钥、启用状态和当前激活 Provider。系统在问答、向量化、OCR 和多模态问答时动态读取当前激活配置，从而避免模型服务被写死在代码中。

```
public ActiveProviderView getActiveProvider(String scope) {
    String normalizedScope = normalizeScope(scope);
    ScopeSettingsView settings = resolveScope(normalizedScope, currentSettings);
    return settings.providers().stream()
            .filter(ProviderConfigView::active)
            .findFirst()
            .map(provider -> toActiveProvider(normalizedScope, provider))
            .orElseThrow(() -> new CustomException(
                    "No active provider configured for scope: " + scope,
                    HttpStatus.INTERNAL_SERVER_ERROR
            ));
}

public synchronized ScopeSettingsView updateScope(String scope,
                                                  UpdateScopeRequ
```

```
est request,
                                                    String updatedBy)
 {
        String normalizedScope = normalizeScope(scope);
        validateUpdateRequest(normalizedScope, request);

        for (ProviderUpsertRequest item : request.providers()) {
            ModelProviderConfig entity = findOrCreateConfig(normalizedScope,
 item.provider());
            entity.setConfigScope(normalizedScope);
            entity.setProviderCode(item.provider());
            entity.setApiBaseUrl(item.apiBaseUrl());
            entity.setModelName(item.model());
            entity.setEnabled(item.enabled());
            entity.setActive(item.provider().equals(request.activeProvider()));
            entity.setUpdatedBy(updatedBy);
            repository.save(entity);
        }

        refreshSettings();
        return resolveScope(normalizedScope, currentSettings);
    }
```

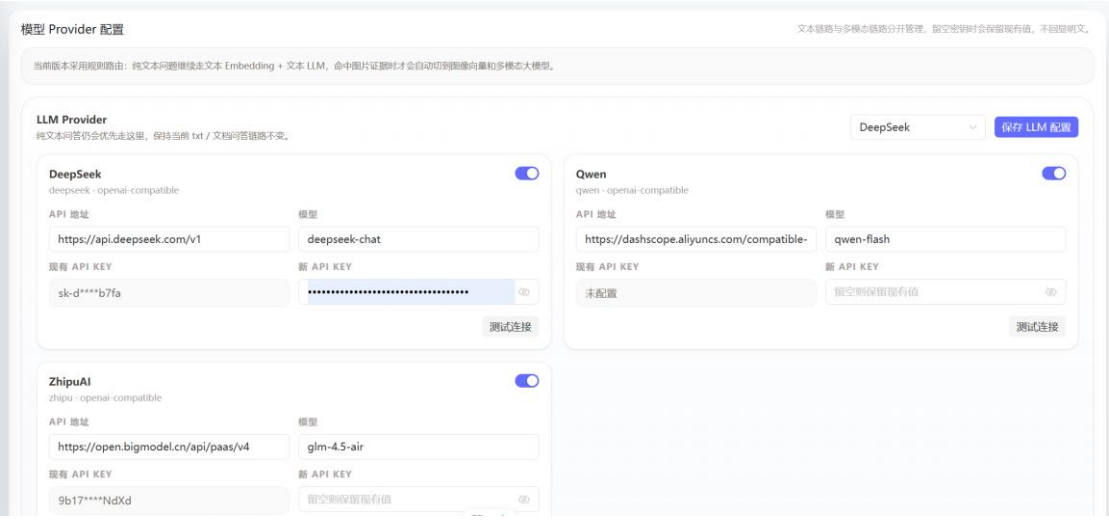

图 5-14 模型配置管理界面图

此处插入模型 Provider 配置页面截图，建议包含 LLM、Embedding、OCR、多模态模型配置。

5.2.9 套餐购买与微信支付模块实现

套餐与支付模块用于实现用户充值、套餐购买和 Token 额度管理。用户选择套餐后，系统创建充值订单，并调用微信支付服务生成扫码支付信息。支付成功后，系统更新订单状态，并为用户增加 LLM Token 和 Embedding Token 额度。

```
public PrePayInfoResBo createOrder(PayOrderReq payReq) {
    String prePayId = createPayOrder(payReq);
    return buildPayInfo(payReq, prePayId);
}

private String createPayOrder(PayOrderReq payReq) {
    PrepayRequest request = new PrepayRequest();
    request.setAppid(wxPayConfig.getAppId());
    request.setMchid(wxPayConfig.getMerchantId());
    request.setDescription(payReq.description());
    request.setNotifyUrl(wxPayConfig.getPayNotifyUrl());
    request.setOutTradeNo(payReq.tradeNo());

    Amount amount = new Amount();
    amount.setTotal(payReq.amount());
    amount.setCurrency("CNY");
    request.setAmount(amount);

    SceneInfo sceneInfo = new SceneInfo();
    sceneInfo.setPayerClientIp(HttpRequestUtil.getClientIp());
    request.setSceneInfo(sceneInfo);

    PrepayResponse response = nativePayService.prepay(request);
    return response.getCodeUrl();
}
```

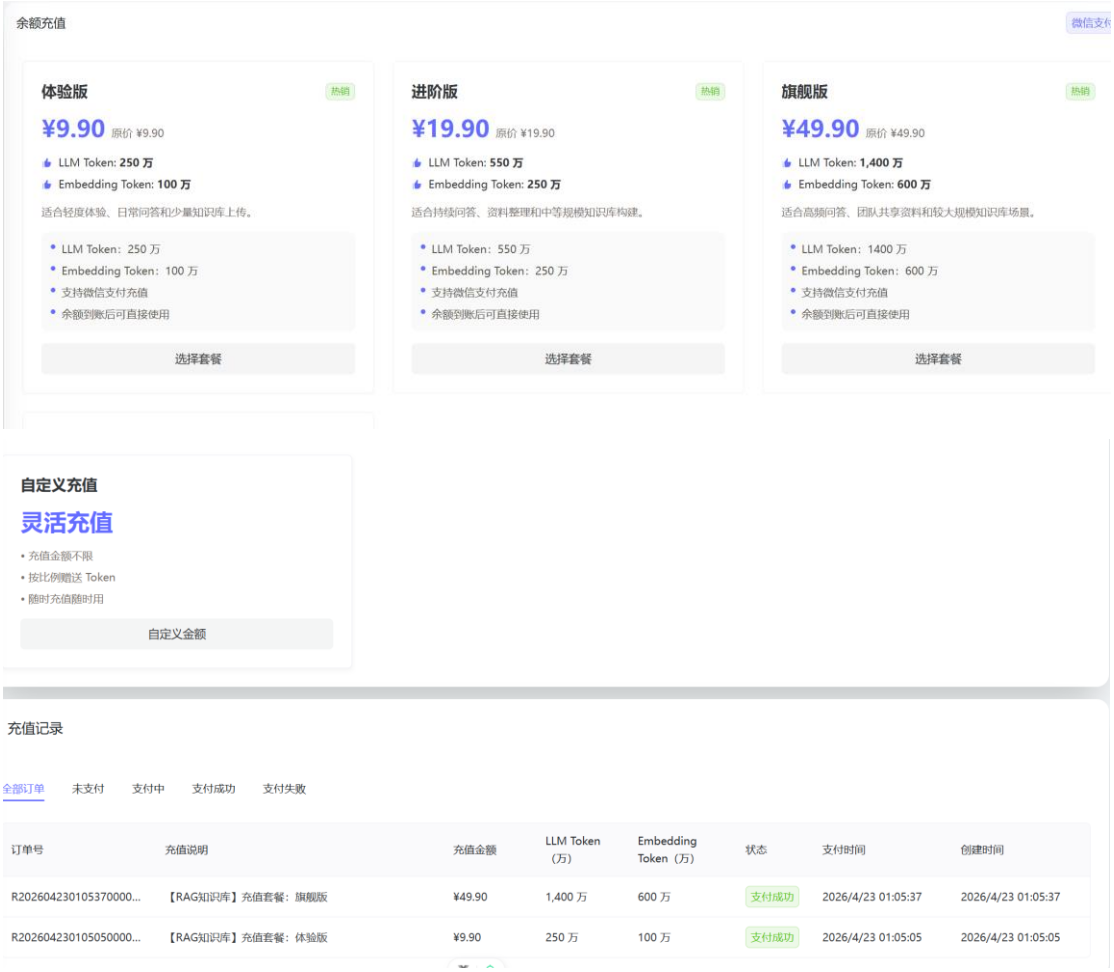

图 5-15 套餐购买界面图

5.2.10 权限管理模块实现

权限管理模块通过用户角色、组织标签和资源可见性共同实现。用户注册后拥有默认组织和私人组织标签；管理员可以为用户分配组织标签；用户上传文件时可以设置资源的组织归属和公开状态。系统在文件列表、知识检索、智能问答等环节都会进行权限过滤。

实现要点如下：

| 权限类型 | 实现方式 |

|---|---|

| 身份认证 | JWT Token |

| 功能权限 | USER、ADMIN 角色区分 |

| 数据权限 | 用户组织标签与文件组织标签匹配 |

| 私有数据 | PRIVATE_username 标签 |

| 公开数据 | is_public 字段 |

| 权限缓存 | Redis 缓存用户有效组织标签 |

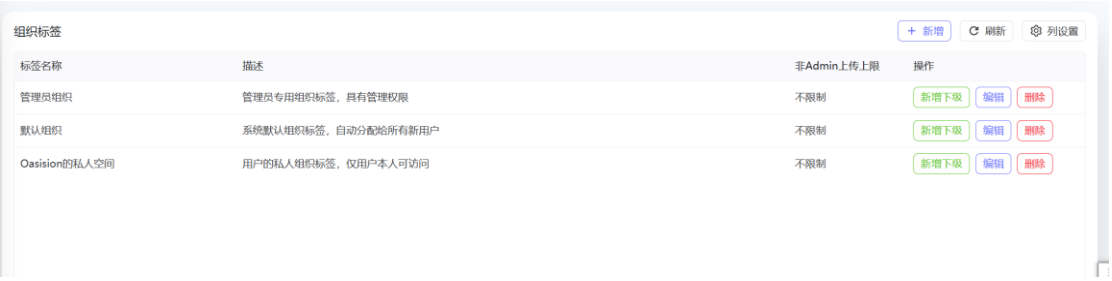

图 5-16 组织标签管理界面图

第六章 系统测试

6.1 测试策略与方法

为验证多模态 RAG 智能知识库系统是否满足功能需求和非功能需求，本文从功能正确性、接口稳定性、权限安全性、异常处理能力和用户交互体验等方面进行测试。测试范围覆盖用户注册登录、邀请码校验、文件上传、OCR 识别、文档解析、向量化入库、知识检索、智能问答、模型配置、套餐购买、微信支付和权限控制等核心模块。

本系统采用以下测试方法：

测试方法	测试说明
黑盒测试	从用户角度验证系统功能是否符合预期，不关注内部代码实现
接口测试	使用接口请求验证后端 API 的参数校验、响应格式和业务结果
功能测试	验证注册登录、文件上传、OCR、问答、支付等功能是否正常
权限测试	验证不同角色、不同组织标签用户的数据访问范围是否正确
异常测试	验证无效参数、重复上传、邀请码失效、额度不足等异常场景
兼容性测试	验证系统在常见浏览器环境下的页面展示和交互效果
性能观察测试	观察文件上传、检索、问答流式返回等关键流程的响应情况

表 6-1 测试策略与方法表

测试过程中，前端主要通过浏览器页面进行操作验证，后端接口通过系统页面、浏览器网络请求和接口调用结果进行检查。对于数据类功能，还需要结合 MySQL、Redis、Elasticsearch 和 MinIO 中的状态变化进行验证，确保系统前端显示、接口返回和后端数据保持一致。

6.2 测试环境

类型	环境
操作系统	Windows
后端运行环境	Java 17、Spring Boot 3.x
前端运行环境	Vue 3、TypeScript、Vite
数据库	MySQL 8.0
缓存	Redis
搜索引擎	Elasticsearch
对象存储	MinIO
消息队列	Kafka
浏览器	Chrome / Edge
测试地址	http://localhost:9527
后端接口地址	http://localhost:8081/api/v1

表 6-2 测试环境表

6.3 测试用例与结果

6.3.1 用户注册与登录测试

测试编号	测试功能	测试步骤	预期结果	实际结果	是否通过
TC-01	用户注册	输入用户名、密码和有效邀请码，点击注册	注册成功，系统创建用户账号	与预期一致	通过
TC-02	邀请码为空注册	不输入邀请码，提交注册	系统提示邀请码不能为空或无效	与预期一致	通过
TC-03	邀请码失效注册	输入已停用或过期邀请码注册	系统拒绝注册并提示邀请码无效	与预期一致	通过
TC-04	重复用户名注册	使用已存在用户名再次注册	系统提示用户名已存在	与预期一致	通过
TC-05	用户登录	输入正确用户名和密码登录	登录成功并跳转系统首页	与预期一致	通过
TC-06	错误密码登录	输入错误密码登录	系统提示用户名或密码错误	与预期一致	通过

表 6-3 用户注册与登录测试表

6.3.2 邀请码管理测试

测试编号	测试功能	测试步骤	预期结果	实际结果	是否通过
TC-07	创建邀请码	管理员填写使用次数和有效期，创建邀请码	邀请码创建成功并显示在列表中	与预期一致	通过
TC-08	批量创建邀请码	管理员输入批量数量并创建	系统生成多条邀请码记录	与预期一致	通过
TC-09	停用邀请码	管理员停用指定邀请码	邀请码状态变为停用，不能继续使用	与预期一致	通过
TC-10	删除邀请码	管理员删除未使用的邀请码	邀请码从列表中移除	与预期一致	通过
TC-11	普通用户访问邀请码管理	普通用户进入邀请码管理接口或页面	系统拒绝访问	与预期一致	通过

表 6-4 邀请码管理测试表

6.3.3 文件上传与解析测试

测试编号	测试功能	测试步骤	预期结果	实际结果	是否通过
TC-12	上传 PDF 文档	选择 PDF 文件上传到知识库	文件上传成功，进入解析流程	与预期一致	通过
TC-13	上传 Word 文档	选择 Word 文件上传	文件上传成功并提取文本	与预期一致	通过
TC-14	上传 TXT 文档	选择 TXT 文件上传	系统成功读取文本内容	与预期一致	通过
TC-15	大文件分片上传	上传较大文件，观察分片上传进度	分片逐步上传，进度正常显示	与预期一致	通过
TC-16	断点续传	上传过程中中断网络或刷新页面后继续上传	系统从未完成分片继续上传	与预期一致	通过
TC-17	重复文件上传	同一用户重复上传相同文件	系统识别文件 MD5，避免重复处理或提示已存在	与预期一致	通过
TC-18	删除文件	用户删除自己上传的文件	文件记录、索引和存储内容被清理	与预期一致	通过

表 6-5 文件上传与解析测试表

6.3.4 OCR 与多模态资料处理测试

测试编号	测试功能	测试步骤	预期结果	实际结果	是否通过
TC-19	上传图片资料	上传包含文字的 PNG/JPG 图片	系统识别图片文字并保存结果	与预期一致	通过
TC-20	上传扫描件	上传扫描版图片或扫描版 PDF	系统通过 OCR 提取文字内容	与预期一致	通过
TC-21	OCR 配置缺失	未配置 OCR 服务时上传图片	系统提示 OCR 未配置或不可用	与预期一致	通过
TC-22	OCR 结果入库	上传图片并等待处理完成	OCR 文本进入切片、向量化和索引流程	与预期一致	通过
TC-23	OCR 权限继承	上传私有图片后用其他用户检索	其他用户无法检索到该图片识别内容	与预期一致	通过

表 6-6 OCR 与多模态资料处理测试表

6.3.5 知识检索测试

测试编号	测试功能	测试步骤	预期结果	实际结果	是否通过
TC-24	关键词检索	输入文档中存在的关键词	系统返回包含关键词的知识片段	与预期一致	通过
TC-25	语义检索	输入与文档表达不同但语义相近的问题	系统返回语义相关的知识片段	与预期一致	通过
TC-26	混合检索	输入包含专业词汇和自然语言的问题	系统综合关键词和语义结果排序	与预期一致	通过
TC-27	topK 参数测试	设置不同 topK 参数进行检索	返回结果数量符合参数限制	与预期一致	通过
TC-28	权限过滤检索	用户检索无权限组织的文档内容	系统不返回无权限内容	与预期一致	通过

表 6-7 知识检索测试表

6.3.6 智能问答测试

测试编	测试功能	测试步骤	预期结果	实际结	是否通

号				果	过
TC-29	普通知识问答	用户输入与知识库相关的问题	系统基于检索内容生成回答	与预期一致	通过
TC-30	多轮对话	用户连续追问同一主题	系统能够结合上下文回答	与预期一致	通过
TC-31	流式输出	用户发起问答后观察页面	回答逐步显示，不需等待完整生成	与预期一致	通过
TC-32	停止生成	回答生成过程中点击停止	系统停止输出并更新状态	与预期一致	通过
TC-33	引用来源展示	查看回答下方引用内容	系统展示相关文档或 OCR 来源	与预期一致	通过
TC-34	无相关知识问答	输入知识库中不存在的问题	系统提示未检索到相关资料或给出合理说明	与预期一致	通过

表 6-8 智能问答测试表

6.3.7 模型配置与模型选择测试

测试编号	测试功能	测试步骤	预期结果	实际结果	是否通过
TC-35	查看模型配置	管理员进入模型配置页面	页面展示 LLM、Embedding、OCR 等配置	与预期一致	通过
TC-36	修改模型配置	管理员修改模型地址或模型名称并保存	系统保存配置并提示成功	与预期一致	通过
TC-37	测试模型连接	点击测试连接按钮	系统返回连接成功或失败信息	与预期一致	通过
TC-38	选择问答模型	用户选择可用模型后发起问答	系统调用用户选择的模型生成回答	与预期一致	通过
TC-39	无权限模型选择	用户选择套餐外模型	系统提示无权限或不可用	与预期一致	通过

表 6-9 模型配置与模型选择测试表

6.3.8 套餐购买与微信支付测试

测试编号	测试功能	测试步骤	预期结果	实际结果	是否通过

TC-40	查看套餐列表	用户进入套餐购买页面	页面展示可购买套餐和权益	与预期一致	通过
TC-41	创建订单	用户选择套餐并点击购买	系统生成订单记录	与预期一致	通过
TC-42	微信扫码支付	用户扫码完成支付	系统更新订单状态为支付成功	与预期一致	通过
TC-43	额度增加	支付成功后查看个人中心	LLM Token 和 Embedding Token 额度增加	与预期一致	通过
TC-44	订单记录查询	用户查看充值订单	系统展示订单金额、状态和时间	与预期一致	通过
TC-45	管理员套餐管理	管理员新增或编辑套餐	套餐配置保存成功并在用户端展示	与预期一致	通过

表 6-10　套餐购买与微信支付测试表

6.3.9 权限控制测试

测试编号	测试功能	测试步骤	预期结果	实际结果	是否通过
TC-46	私有文件访问	用户上传私有文件，其他用户访问	其他用户无法查看或检索	与预期一致	通过
TC-47	公开文件访问	用户上传公开文件	其他用户可查看或检索该文件内容	与预期一致	通过
TC-48	组织文件访问	用户上传组织文件	同组织用户可访问，非同组织用户不可访问	与预期一致	通过
TC-49	管理员访问	管理员查看用户、文件和订单信息	管理员可正常访问管理功能	与预期一致	通过
TC-50	普通用户访问后台接口	普通用户请求管理员接口	系统返回无权限或拒绝访问	与预期一致	通过

表 6-10　权限控制测试表

第七章 总结与展望

7.1 工作总结

本项目围绕企业知识管理中资料分散、检索效率不高、图片和扫描件资料难以利用等问题，这导致场景文本关系推理不足，模型可能对场景文本的描述不准确。另一个可能的原因是，现有方法无法生成某些视觉对象的细粒度描述[15]，因此设计并实现了一套基于多模态 RAG 的智能知识库系统。系统以 Spring Boot 和 Vue 3 为主要开发框架，结合 MySQL、Redis、Elasticsearch、Kafka 和 MinIO 等技术，实现了从资料上传、文档解析、OCR 识别、文本切片、向量化入库到智能问答的完整流程。

在功能实现方面，系统支持用户注册登录、邀请码准入、组织标签权限控制、知识库管理、文件分片上传、断点续传、OCR 文字识别、混合检索、WebSocket 流式问答、聊天历史保存、模型配置与模型选择、套餐购买和微信支付等功能。用户可以上传 Word、PDF、TXT、图片、截图和扫描件等资料，系统会根据文件类型进行解析或 OCR 识别，并将处理后的文本内容转化为知识片段，用于后续语义检索和 RAG 问答。短查询处理：主要依靠向量相似度检索和语义召回简短查询关键词信息有限，向量检索能更好捕捉语义意图通常设置较小的 top-k 值(如 10-15)提高检索精度长查询处理：采用混合检索策略，结合向量相似度检索和 BM25 关键词检索详细查询包含更多明确关键词，关键词检索提供更精确匹配适当增加 top-k 值(如 20-30)获取更全面上下文。我们的方法利用基于 BM25 的文本标记，在保持检测性能的同时显著降低了计算成本[13]。

在智能问答方面，系统采用检索增强生成方式。用户提出问题后，系统先从知识库中召回相关内容，再结合历史对话和检索结果构造 Prompt，调用大语言模型生成回答，并展示引用来源。该方式使回答能够更好地结合用户已有资料，也提高了问答结果的可追溯性。

在系统管理方面，本文实现了模型配置和模型选择功能，使管理员能够维护 LLM、Embedding、OCR、视觉向量和多模态模型等不同类型的模型配置；同时通过邀请码系统控制用户注册入口，通过套餐和微信支付模块管理用户额度与服务权益。这些功能提高了系统的可配置性和可运营性。

通过系统测试可以看出，系统主要功能运行基本符合预期，能够完成多模态资料入库、知识检索、智能问答、权限控制和套餐支付等核心业务流程。总体而言，本文完成了一个较完整的企业智能知识库系统设计与实现，为 RAG、OCR 和多模态资料处理在知识管理场景中的应用提供了一种可行方案。

7.2 未来展望

虽然系统已经实现了主要功能，但仍有进一步优化空间。

首先，在 OCR 和多模态理解方面，后续可以继续提升图片、扫描件和复杂版面文档的识别效果。例如针对表格、流程图、代码截图和带有复杂排版的扫描件，可以引入更强的版面分析和图文理解模型，使系统不仅能提取文字，还能更好地理解图文之间的结构关系。

其次，在检索效果方面，当前系统主要基于 Elasticsearch 实现关键词检索与向量检索。后续可以进一步优化混合检索的权重策略，引入重排序模型，提高检索结果的相关性。同时，随着知识库规模扩大，可以考虑接入 FAISS 等专用向量检索引擎，以提升大规模向量检索性能。

再次，在智能问答方面，后续可以优化 Prompt 模板管理和上下文压缩策略，使系统在多轮对话中更好地保留关键信息，减少无关内容干扰。同时，可以增加回答质量评估机制，对回答是否引用知识库、是否存在不确定内容等进行辅助判断。

此外，在模型管理方面，系统可以继续完善不同模型之间的调用策略，例如根据问题类型自动选择文本模型、多模态模型或 OCR 相关模型；也可以根据套餐等级、调用成本和用户额度动态选择合适模型，从而在效果和成本之间取得更好的平衡。

在系统运营方面，后续可以进一步完善套餐管理、订单管理、用量统计和费用分析功能，使管理员能够更清晰地掌握用户使用情况和模型调用成本。同时，可以增加更细粒度的数据统计报表，为系统后续运营和优化提供依据。

最后，在安全和稳定性方面，后续可以加强数据备份、异常任务重试、支付回调校验、敏感信息加密和操作日志审计等能力，进一步提升系统在真实企业环境中的可靠性和安全性。总体来看，本系统仍可以在多模态理解、检索性能、问答质量、模型调度和运营管理等方面持续优化。

参考文献

[1] 康智峰,张亚生.基于微调与检索增强生成的混合式智能协议理解方法[J]. 计算机测量与控制,2026,34(04):265-271.DOI:10.16526/j.cnki.11-4762/tp.2026.04.033.

[2] 李泽鸣,王树良,尚子贺,等.多模态检索增强生成驱动的文档问答综述(特邀) [J].计算机工程,2026,52(04):1-21.DOI:10.19678/j.issn.1000-3428.0260043.

[3] Ye N . Elasticsearch for Complex Data Association Analysis: Modeling, Aggregation, and Optimization Techniques [J]. Frontiers in Computing and Intelligent Systems, 2025, 12 (3): 5-11. DOI:10.54097/32W55K51.

[4] Zhou X ,Li S ,Liu Y , et al.LLM-based domain-specific knowledge graph construction and hybrid retrieval-augmented generation[J].Journal of Manufacturing Systems,2026,861022-1038.DOI:10.1016/J.JMSY.2026.04.023.

[5] 肖鹏程.基于 ElasticSearch 与 LCS 的文本相似度计算方法研究[D].四川师范大学,2024.DOI:10.27347/d.cnki.gssdu.2024.000577.

[6] Harris S ,Ta T V ,Trovati M , et al.Multimodal misinformation detection across diverse languages using RAG and LLMs[J].Journal of Intelligent Information Systems,2026,(prepublish):1-28.DOI:10.1007/S10844-026-01042-X.

[7] Zhou X ,Li S ,Liu Y , et al.LLM-based domain-specific knowledge graph construction and hybrid retrieval-augmented generation[J].Journal of Manufacturing Systems,2026,861022-1038.DOI:10.1016/J.JMSY.2026.04.023.

[8] Muthusami R ,Saritha K .Hybrid retrieval generation for structured reasoning with large language models[J].Discover Artificial Intelligence,2026,6(1):335-335.DOI:10.1007/S44163-026-01059-9.

[9] Kiss C ,Nagy M ,Szilágyi P .Max–Min semantic chunking of documents for RAG application[J].Discover Computing,2025,28(1):117-117.DOI:10.1007/S10791-025-09638-7.

[10] Tran K Q ,Pham K N .Enhancing Segment-Based Bag of Clusters with Mixture Models and Late Chunking for Document Clustering[J].SN Computer Science,2025,6(7):897-897.DOI:10.1007/S42979-025-04411-5.

[11] Abdolinejad F ,Eftekhari M .Augmenting RAG with nonnegative matrix factorization-driven semantic chunking in embedding space[J].The Journal of Supercomputing,2026,82(4):224-224.DOI:10.1007/S11227-026-08370-3.

[12] Cediel M A ,Lopez G E ,Cabot G A , et al.Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries[J].Knowledge-Based Systems,2025,331114896-114896.DOI:10.1016/J.KNOSYS.2025.114896.

[13] Kim W J ,Choi J M .A cross-language and cross-binary type approach to binary-source software composition analysis using BM25[J].International Journal of Information Security,2025,24(6):231-231.DOI:10.1007/S10207-025-01148-3.

[14] 李子骏,肖辉,李雪峰.面向知识密集型任务的检索增强生成技术综述[J].微电子学与计算机,2025,42(10):48-65.DOI:10.19304/J.ISSN1000-7180.2025.0652.

[15] Xu D ,Huang Q ,Zhang X , et al.DEVICE: Depth and Visual Concepts Aware Transformer for OCR-based image captioning[J].Pattern Recognition,2025,164111522-111522.DOI:10.1016/J.PATCOG.2025.111522.

[16] Liu J ,Huang Y ,Tan J , et al.Enhancing document retrieval using semantic alignment with hierarchical graph matching[J].The Journal of Supercomputing,2026,82(5):339-339.DOI:10.1007/S11227-026-08490-W.

[17] Saleem S ,Asim N M ,Zulfiqar S , et al.The evolution of natural language processing: How prompt optimization and language models are shaping the future[J].Computer Science Review,2026,61100938-100938.DOI:10.1016/J.COSREV.2026.100938.

[18] 李军佳,蔡国永.基于混合检索与提示词优化的增强生成框架及其应用[J].计算机时代,2026,(01):40-46.DOI:10.16644/j.cnki.cn33-1094/tp.2026.01.008.

[19] 卢万有.基于 JWT 的 RBAC 在前后端分离项目中的设计与实现[J].电脑编程技巧与维护,2025,(01):46-48.DOI:10.16184/j.cnki.comprg.2025.01.004

[20] Alawwad A H ,Alhothali A ,Naseem U , et al.Enhancing textual textbook question answering with large language models and retrieval augmented generation[J].Pattern Recognition,2025,162111332-111332.DOI:10.1016/J.PATCOG.2024.111

致谢

执笔至此，我四年的本科学习即将结束。回首这段从年轻到逐渐成熟的旅程，我既不情愿，也心存感激。在大学的四年里，我在课堂上获得了知识，通过实践锻炼了自己的能力，并通过反复的选择、尝试和坚持不断了解自己。毕业设计和论文的完成不仅是对我专业学习成果的总结，也是对我大学期间学习能力、实践能力和思维能力的全面考验。在此，我谨向所有关心、帮助和支持我的老师、家人、同学和朋友表示最诚挚的感谢。

首先，我要衷心感谢我的母校昌大。学校为我提供了良好的学习环境、扎实的专业课程和丰富的实践平台，使我在四年的学习中不断积累和成长。我也非常感谢学校和企业共同搭建的校企合作平台，这让我有机会走出课堂，进入真实的企业环境，了解软件开发、项目协作和工程实施在实践中的意义。这段经历让我更加清楚自己的职业方向，也让我对未来的学习和工作有了更清晰的了解。

感谢我的论文导师林教授。在我撰写毕业设计和论文的过程中，林老师给予了我耐心细致的指导。同时，我衷心感谢辅导员詹宇老师。在大学期间，詹宇教授不仅在我的日常校园生活中给予我关怀和帮助，而且对我的价值观、思维方式和成长方向产生了重大影响。很多时候，老师的提醒和指导教会了我以更成熟、更理性的方式面对学习、生活和未来的选择。感谢詹瑜老师一路的陪伴和指导。这种影响将在我的成长道路上持续很长一段时间。

感谢校企合作负责人万老师，感谢学校和企业为我提供学习和实习机会。正是这个宝贵的平台和资源，使我能够将课堂上学到的知识与实际的企业经验相结合，并在真实的项目环境中体验技术的价值和责任。特别感谢我在恒生电子实习期间的导师刘同格。刘同格在具体的生产线工作中为我提供了很多耐心的指导，不仅让我学习了实用的开发方法、技能和标准，还让我了解了团队合作、问题定位和工程质量的重要性。这次实习经历是我大学生活中非常宝贵的一部分，为我完成毕业设计提供了重要的灵感。

感谢我的室友、同学和朋友。在大学的四年里，我们一起上课，讨论问题，准备考试，一起面对压力，调整状态，互相鼓励。那些共同工作的日子，那些在宿舍、教室和校园里的平凡时刻，现在回想起来尤其珍贵。正是因为有了你的陪伴，许多紧张和困惑的时刻变得不那么孤独。谢谢你带给我温暖、支持和力量。

最后，我要向我的父母表示最深切的感谢。在整个旅程中，你一直是我最坚定的支持。当我遇到困难时给予理解，当我感到失落时给予鼓励，当我想继续前进时给予支持。你的宽容、信任和奉献使我能够安心学习，并给了我面对未来的

勇气。在每一步成长的背后，我们离不开您的默默保护。

此时，我的大学生活即将成为回忆，但这段经历带来的收获和情感将永远伴随着我的旅程。感谢所有的遭遇，感谢所有的帮助，感谢你们一路上的辛勤工作。前方的路还很长，带着这份感激和爱，我将继续保持我的初衷，认真对待生活的每一步，努力走向更广阔的世界。